

Manual Técnico

Sistema GEA — Gestión de Eventos y Anuncios
Universidad Libre, Seccional Cúcuta

Versión 1.0 · 2026

0. Introducción

0.1 Propósito de este manual

Este manual técnico es la referencia integral de la plataforma **GEA (Gestión de Eventos y Anuncios)** de la Universidad Libre, Seccional Cúcuta. Está dirigido a **desarrolladores y personal técnico** que se incorporan al proyecto y necesitan comprender, mantener, extender o desplegar el sistema sin contar con conocimiento previo de su historia o de las decisiones de diseño tomadas.

El documento cubre **de forma exhaustiva los tres sistemas** que componen la plataforma: el **backend** (API REST en Spring Boot), el **frontend web** (panel administrativo en React) y la **aplicación móvil** (cliente en Flutter). Para cada uno se describe su estructura de código, sus dependencias, su modelo de datos, los flujos de negocio que implementa y los procedimientos de arranque, configuración y despliegue. También se documenta la **integración entre sistemas** y con los servicios externos (correo institucional, notificaciones push y autenticación corporativa).

El manual está organizado en partes secuenciales, pensadas para leerse en orden por un perfil nuevo pero también para consultarse de manera puntual:

Parte	Contenido
0. Introducción	Visión general, arquitectura global, stack tecnológico y glosario.
1. Arranque	Requisitos previos y puesta en marcha del entorno de desarrollo.
2. Backend	Estructura, capas, seguridad, modelo de datos y API del backend Spring Boot.
3. Frontend	Panel administrativo web en React.
4. App Flutter	Aplicación móvil, arquitectura limpia y manejo de estado con Riverpod.
5. Integración	Comunicación entre sistemas y servicios externos.
6. Despliegue	Procedimientos de build y publicación de cada sistema.
7. Troubleshooting	Diagnóstico de problemas frecuentes.
8. Anexos	Material de referencia complementario.

0.2 ¿Qué es GEA?

GEA es una **plataforma institucional para la gestión, aprobación y publicación de eventos y anuncios** de la Universidad Libre, Seccional Cúcuta. Su objetivo es centralizar el ciclo de vida de toda comunicación institucional —desde que una dependencia la solicita hasta que se publica y se difunde a la comunidad universitaria— sustituyendo procesos manuales y dispersos por un flujo controlado, auditable y con trazabilidad.

Funcionalmente, GEA ofrece:

- **Calendario público de eventos**, accesible desde la aplicación móvil y desde el navegador, que muestra los eventos ya publicados de la institución.
- **Difusión de anuncios** institucionales (comunicados, noticias, piezas gráficas) hacia los usuarios de la app.
- **Flujo de aprobación multi-rol**: las **oficinas** (dependencias académicas y administrativas) crean *solicitudes* de eventos o anuncios; el área de **Comunicaciones** y los **administradores** las revisan, aprueban o rechazan y, finalmente, las **publican**. Cada solicitud transita por estados bien definidos (**PENDIENTE** → **APROBADA** → **PUBLICADA**, o **RECHAZADA**), lo que garantiza control editorial y separación de responsabilidades.
- **Notificaciones** automáticas por correo electrónico institucional y notificaciones *push* a los dispositivos móviles cuando se publican contenidos de interés.
- **Reportes** sobre la actividad de la plataforma (solicitudes, publicaciones, eventos por oficina, etc.), exportables a PDF y Excel.
- **Auditoría** de los cambios sobre las entidades del dominio, registrada de forma histórica.

Los **usuarios** de la plataforma se agrupan en perfiles con distintos niveles de acceso. Los roles administrativos (**SUPER_ADMIN**, **ADMIN**, **COMUNICACIONES**, **OFICINA**) operan sobre el panel web; los usuarios finales de la app pueden ser anónimos (**USUARIO_APP**) o autenticados (**USUARIO_AUTENTICADO_APP**) cuando inician sesión con sus credenciales institucionales. La gestión de eventos, anuncios, oficinas y lugares físicos se realiza desde el panel administrativo, mientras que la app móvil es principalmente un cliente de consumo y consulta para la comunidad universitaria.

0.3 Arquitectura global

GEA está construido como un sistema **cliente-servidor de tres capas** con un backend central y múltiples clientes. El **backend Spring Boot** es el núcleo: expone una **API REST** sobre HTTP/HTTPS y concentra toda la lógica de negocio, la seguridad y el acceso a datos. Es el único componente que se comunica con la base de datos y con los servicios externos; los clientes nunca acceden directamente a la persistencia.

Existen tres tipos de **clientes** que consumen la misma API:

- El **panel web en React**, usado por los roles administrativos para gestionar el contenido y los catálogos.
- La **aplicación móvil en Flutter** (Android/iOS), orientada a la comunidad universitaria.

- El acceso vía **navegador (PWA)**, que reutiliza la capa web para consulta del calendario y anuncios públicos.

Toda la comunicación entre clientes y backend se realiza mediante **peticiones REST sobre HTTP/HTTPS**, autenticadas con **JSON Web Tokens (JWT)**. El backend escucha en el puerto **8083** con el *context-path* **/api**, de modo que todos los recursos cuelgan de la ruta base `http://<host>:8083/api`.

La persistencia se apoya en una **base de datos MySQL central** (`gea_db`), accedida mediante JPA/Hibernate con un pool de conexiones HikariCP y versionada con migraciones Flyway. El historial de cambios se conserva mediante Hibernate Envers (tablas de auditoría con sufijo `_audit`).

El backend se integra además con **tres servicios externos**:

- **SMTP institucional** — envío de correos de notificación a oficinas y destinatarios configurados.
- **Firebase Cloud Messaging (FCM)** — entrega de notificaciones *push* a los dispositivos móviles registrados.
- **Microsoft Azure AD** — autenticación corporativa (SSO) para el inicio de sesión institucional de usuarios autenticados.

La Figura 1 resume esta arquitectura y las relaciones entre componentes.

Figura 1: Arquitectura global del sistema GEA

Figura 1: Arquitectura global del sistema GEA

0.4 Stack tecnológico

Las tablas siguientes detallan, por sistema, las tecnologías principales con su versión y propósito. Las versiones provienen de los archivos de configuración reales de cada repositorio (`pom.xml`, `package.json` y `pubspec.yaml`).

Backend (Spring Boot)

Tecnología	Versión	Propósito
Java	21	Lenguaje y plataforma de ejecución.
Spring Boot	3.4.3	Framework base (web, seguridad, JPA, mail, cache, actuador).
Spring Security + OAuth2 Resource Server	3.4.3	Autenticación/autorización y validación de tokens.
JWT	0.11.5	Generación y verificación de JSON Web Tokens.
Spring Data JPA / Hibernate	3.4.3	Mapeo objeto-relacional y acceso a datos.
Hibernate Envers	3.4.3	Auditoría histórica de entidades.
MySQL Connector/J	runtime	Conector JDBC a la base de datos MySQL.
Flyway (core + mysql)	gestionado por Spring Boot	Migraciones versionadas de esquema.
HikariCP	gestionado por Spring Boot	Pool de conexiones a base de datos.
Caffeine	gestionado por Spring Boot	Caché en memoria.
Firebase Admin SDK	9.2.0	Envío de notificaciones push (FCM).
MapStruct	1.5.5.Final	Mapeo entre entidades y DTOs.
Lombok	1.18.34	Reducción de código repetitivo (boilerplate).
springdoc-openapi	2.6.0	Documentación interactiva de la API (Swagger UI).
OpenPDF / Apache POI / JFreeChart	1.3.39 / 5.3.0 / 1.5.5	Generación de reportes (PDF, Excel y gráficos).

Frontend web (React)

Tecnología	Versión	Propósito
React / React DOM	19.2.4	Librería de interfaz de usuario.
Vite	8.0.1	Empaquetador y servidor de desarrollo.
React Router DOM	7.13.1	Enrutamiento del panel administrativo.
Axios	1.13.6	Cliente HTTP para consumir la API REST.
Recharts	3.8.1	Gráficos y visualización de reportes.
Lucide React	0.577.0	Iconografía.
React Toastify	11.0.5	Notificaciones en interfaz.
ESLint	9.39.4	Análisis estático y calidad de código.

App móvil (Flutter)

Tecnología	Versión	Propósito
Flutter / Dart SDK	Dart ^3.11.5	Framework y lenguaje de la app multiplataforma.
flutter_riverpod / riverpod_annotation	2.5.1 / 2.3.5	Manejo de estado y lógica de presentación.
go_router	13.2.0	Navegación declarativa.
dio	5.4.1	Cliente HTTP para la API REST.
flutter_secure_storage	9.0.0	Almacenamiento seguro de tokens.
shared_preferences	2.2.2	Persistencia local de preferencias.
aad_oauth	1.0.1	Inicio de sesión con Microsoft Azure AD.
connectivity_plus	6.1.0	Detección de conectividad (modo offline).
table_calendar	3.1.0	Vista de calendario de eventos.
intl	0.20.2	Internacionalización y formato de fechas.
fpdart / equatable	0.6.0 / 2.0.5	Programación funcional y comparación de valores.
cached_network_image / flutter_svg	3.4.1 / 2.0.10	Carga y renderizado de imágenes y SVG.

0.5 Glosario

Término	Definición
Solicitud	Petición creada por una oficina para publicar un evento o un anuncio. Es el punto de entrada del flujo de aprobación y atraviesa los estados PENDIENTE, APROBADA, RECHAZADA o PUBLICADA. Se materializa en las entidades <code>SolicitudEvento</code> y <code>SolicitudAnuncio</code> .
Publicación	Contenido ya aprobado y difundido a la comunidad. Resulta de publicar una solicitud aprobada. Se materializa en <code>PublicacionEvento</code> y <code>PublicacionAnuncio</code> .
Evento	Actividad institucional con fecha, lugar y participantes (charla, ceremonia, taller, etc.) que se muestra en el calendario público una vez publicada.
Anuncio	Comunicado o noticia institucional difundida a los usuarios de la app, habitualmente acompañado de una pieza gráfica.
Oficina	Dependencia académica o administrativa de la universidad que origina solicitudes. Entidad <code>Oficina</code> ; rol asociado OFICINA .
Pieza Gráfica	Recurso visual (imagen) que acompaña un anuncio o evento; gestionado como archivo adjunto (<code>ArchivoAdjunto</code>).
Lugar Físico	Espacio donde se realiza un evento (auditorio, aula, etc.). Entidad <code>LugarFísico</code> .
SIAPAC	Sistema institucional de información académico-administrativo de la Universidad Libre con el que se relacionan datos de usuarios y dependencias.
Rol	Perfil de autorización del usuario. Valores: SUPER_ADMIN, ADMIN, COMUNICACIONES, OFICINA, USUARIO_APP, USUARIO_AUTENTICADO_APP (y USUARIO como rol base). Enum <code>Rol</code> .
Estado	Fase del ciclo de vida de una solicitud (enum <code>EstadoSolicitud</code>): PENDIENTE (recién creada, a la espera de revisión), APROBADA (validada, lista para publicar), PUBLICADA (visible para la comunidad) y RECHAZADA (descartada por el revisor).
JWT (JSON Web Token)	Token firmado que transporta la identidad y los permisos del usuario en cada petición a la API; base de la autenticación sin estado del backend.
Riverpod	Librería de manejo de estado de Flutter usada en la app para la capa de presentación (providers y notifiers).
Clean Architecture	Estilo arquitectónico que separa estrictamente UI, lógica de presentación y capa de datos/dominio. Es la arquitectura de referencia de la app móvil.
DTO (Data Transfer Object)	Objeto plano que transporta datos entre el cliente y la API, desacoplado de las entidades de persistencia. El mapeo entidad ↔ DTO se realiza con MapStruct.
FCM (Firebase Cloud Messaging)	Servicio de Google para el envío de notificaciones push a los dispositivos móviles registrados.
Azure AD	Microsoft Entra/Azure Active Directory; proveedor de identidad institucional usado para el inicio de sesión corporativo (SSO).
Flyway	Herramienta de migraciones versionadas que mantiene el esquema de la base de datos sincronizado y reproducible.
Envers	Módulo de Hibernate que registra el historial de cambios de las entidades en tablas de auditoría (sufijo <code>_audit</code>).
Actuator	Módulo de Spring Boot que expone endpoints operativos (health, info, metrics) para monitoreo del backend.

1. Guía de arranque

Esta parte describe cómo poner en marcha **los tres sistemas de GEA en un entorno de desarrollo local**: el backend (Spring Boot), el frontend web (React + Vite) y la aplicación móvil (Flutter). Todos los valores indicados están extraídos directamente de los archivos de configuración reales de cada repositorio.

1.1 Requisitos previos

Antes de clonar y arrancar los proyectos, asegúrate de tener instaladas las siguientes herramientas. Las versiones provienen de `pom.xml`, `package.json` y `pubspec.yaml`.

Herramienta	Versión requerida	Propósito
JDK (Java)	21 (<code>java.version=21</code> en <code>pom.xml</code>)	Compilar y ejecutar el backend Spring Boot 3.4.3.
Maven	No es necesario instalarlo: el repo incluye el <code>wrapper mvnw / mvnw.cmd</code>	Gestión de dependencias y build del backend.
Node.js	18 o superior (el entorno de referencia usa Node v22)	Ejecutar Vite y el panel web. Vite 8 requiere Node moderno.
npm	El que acompaña a Node	Instalar dependencias del frontend.
Flutter SDK	Dart ^3.11.5 (<code>environment.sdk</code> en <code>pubspec.yaml</code>)	Compilar y ejecutar la app móvil.
MySQL	8.x	Base de datos del backend (<code>gea_db</code>).
Git	Cualquier versión reciente	Clonar los repositorios.
Google Chrome (opcional)	—	Sólo necesario para regenerar el PDF de este manual (<code>build.js</code>).

Nota: el backend usa el conector `mysql-connector-j` y el dialecto de MySQL 8. No se utiliza H2 en ejecución; H2 sólo está declarado con `scope=test` para pruebas de integración en memoria.

1.2 Estructura de repositorios

GEA se compone de tres repositorios independientes. Las rutas locales corresponden al entorno de desarrollo de referencia.

Sistema	Repositorio GitHub	Ruta local de referencia
Backend (API REST)	x6Darck/GEA_BACKEND	C:\Users\Administrador\Documents\test\GEA_BACKEND
Frontend (panel web)	x6Darck/GEA_FRONT	C:\Users\Administrador\Documents\Proyecto 2\GEA_FRONT
App móvil (Flutter)	x6Darck/GEA_MOVIL	C:\Users\Administrador\Documents\gea_app

La ruta local es indicativa; puedes clonar cada repositorio donde prefieras. Lo importante es respetar la configuración de puertos y variables que se describe a continuación.

1.3 Backend — levantar en local

El backend es una aplicación Spring Boot que expone la API REST bajo el `context-path /api` en el puerto **8083**, con un puerto de administración (Actuator) separado en el **8084**.

Pasos:

1. Clonar el repositorio:

```
git clone https://github.com/x6Darck/GEA_BACKEND.git
cd GEA_BACKEND
```

2. Crear la base de datos en tu servidor MySQL local. El nombre por defecto es `gea_db` (definido en `DB_URL`):

```
CREATE DATABASE gea_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

La URL de conexión por defecto es: `jdbc:mysql://localhost:3306/gea_db?`

`useSSL=false&serverTimezone=America/Bogota&allowPublicKeyRetrieval=true&characterEncoding=UTF-8`

3. **Configurar las credenciales de la base de datos.** Por defecto el backend usa usuario `root` y contraseña `1234`. Si tu MySQL local difiere, puedes:

- Definir las variables de entorno `DB_USERNAME` y `DB_PASSWORD` (recomendado), o
- Editar directamente `src/main/resources/application.properties`.

En PowerShell (Windows):

```
$env:DB_USERNAME = "root"
$env:DB_PASSWORD = "tu_password"
```

4. **Levantar la aplicación** con el wrapper de Maven incluido en el repo. En Windows usa `mvnw.cmd`:

```
.\mvnw.cmd spring-boot:run
```

En Linux/macOS:

```
./mvnw spring-boot:run
```

Durante el arranque ocurre lo siguiente automáticamente:

- **Flyway** ejecuta las migraciones de `classpath:db/migration` (la migración `baseline v1__baseline_schema.sql`). El esquema queda versionado; `baseline-on-migrate=true` permite tomar como base una BD ya existente.
- Hibernate arranca con `ddl-auto=validate`, es decir **valida** el esquema contra las entidades pero **no lo modifica** (las migraciones las hace Flyway, no Hibernate).
- Los componentes `DataInitializer` y `OficinaDataSeeder` (en `config/`) siembran los datos iniciales (usuarios/roles base y catálogo de oficinas).

5. **Verificar que el backend responde:**

- API REST: `http://localhost:8083/api`
- Documentación Swagger UI: `http://localhost:8083/api/swagger-ui.html`
- Health check (Actuator, puerto de management 8084): `http://localhost:8084/actuator/health`

El servidor escucha en `server.address=0.0.0.0`, por lo que también es accesible desde otros dispositivos de la red local mediante la IP de la máquina (necesario para probar la app móvil en un teléfono físico — ver 1.5).

1.4 Frontend — levantar en local

El panel administrativo está construido con **React 19** y **Vite 8**. La URL del backend se inyecta mediante la variable `VITE_API_URL`.

Pasos:

1. **Clonar y entrar al directorio:**

```
git clone https://github.com/x6Darck/GEA_FRONT.git
cd GEA_FRONT
```

2. **Instalar dependencias:**

```
npm install
```

3. **Crear el archivo** `.env.local` en la raíz del proyecto apuntando al backend local:

```
VITE_API_URL=http://localhost:8083
```

Cómo se usa: en `src/services/api.js` la instancia de Axios construye su `baseUrl` como ``${API_URL}/api`` a partir de `import.meta.env.VITE_API_URL`, con `fallback` a `http://localhost:8083` si la variable no está definida. Es decir, **se indica sólo el host y el puerto; el sufijo `/api` lo añade el cliente automáticamente** (coincidiendo con el `context-path` del backend).

Para el entorno productivo existe `.env.production` con `VITE_API_URL=https://api.gea.tudominio.com`. No lo uses en desarrollo local.

4. Arrancar el servidor de desarrollo:

```
npm run dev
```

Vite levanta el panel en `http://localhost:5173`. Este origen ya está incluido en la lista CORS por defecto del backend (`CORS_ALLOWED_ORIGINS`), por lo que las peticiones funcionan sin configuración adicional.

Otros scripts disponibles (`package.json`): `npm run build` (build de producción), `npm run preview` (servir el build) y `npm run lint` (ESLint).

1.5 App Flutter — levantar en local

La aplicación móvil usa **Flutter (Dart ^3.11.5)**, **Riverpod** para el estado y **GoRouter** para la navegación. La URL del backend se inyecta en tiempo de compilación con `--dart-define=API_BASE_URL=...`.

Pasos:

1. Obtener las dependencias:

```
flutter pub get
```

2. Generar las localizaciones (el proyecto declara `flutter: generate: true` y usa `flutter_localizations` + `intl`):

```
flutter gen-l10n
```

3. Ejecutar la app indicando la URL del backend. En un **emulador o dispositivo físico** debes usar la **IP de la PC** que ejecuta el backend, no `localhost` (en el teléfono `localhost` apunta al propio teléfono):

```
flutter run --dart-define=API_BASE_URL=http://<IP_DE_TU_PC>:8083/api
```

Ejemplo, si tu PC tiene la IP `192.168.1.20`:

```
flutter run --dart-define=API_BASE_URL=http://192.168.1.20:8083/api
```

Cómo se usa: en `lib/core/config/app_config.dart`, `AppConfig.apiUrl` se lee con `String.fromEnvironment('API_BASE_URL', ...)`. Si no pasas el `--dart-define`, se aplica el valor por defecto `http://172.46.12.57:8083/api`. **A diferencia del frontend web, aquí sí debes incluir el sufijo `/api` en la URL.**

Recuerda que el backend escucha en `0.0.0.0:8083`, lo que permite que el teléfono lo alcance por la IP de la PC siempre que ambos estén en la misma red y el firewall lo permita.

4. Builds de producción (referencia, ver comentario en `app_config.dart`):

```
flutter build apk --dart-define=API_BASE_URL=https://api.gea.tudominio.com/api
```

1.6 Variables de entorno (referencia completa)

Tabla maestra de todas las variables configurables. Para el backend, cada fila corresponde a un patrón `${VAR:valor_por_defecto}` de `application.properties`.

Backend (`application.properties`)

Variable	Valor por defecto	Descripción
DB_URL	<code>jdbc:mysql://localhost:3306/gea_db?useSSL=false&serverTimezone=America/Bogota&allowPublicKeyRetrieval=true&characterEncoding=UTF-8</code>	URL JDBC de la base de datos MySQL.
DB_USERNAME	<code>root</code>	Usuario de la base de datos.
DB_PASSWORD	<code>1234</code>	Contraseña de la base de datos.
DB_POOL_MAX	<code>20</code>	Tamaño máximo del pool de conexiones HikariCP (<code>maximum-pool-size</code>).
DB_POOL_MIN	<code>5</code>	Conexiones mínimas en reposo del pool (<code>minimum-idle</code>).
DDL_AUTO	<code>validate</code>	Estrategia DDL de Hibernate. En producción usar <code>validate</code> o <code>none</code> ; <code>update</code> puede corromper datos.
FLYWAY_ENABLED	<code>true</code>	Activa/desactiva las migraciones Flyway al arranque.
JWT_SECRET	<code>dev_only_change_this_secret_in_production_min32chars</code>	Secreto de firma de los JWT. Mínimo 32 caracteres. Nunca usar el valor por defecto en producción.
CORS_ALLOWED_ORIGINS	<code>http://localhost:5173,http://localhost:3000</code>	Orígenes permitidos por CORS (lista separada por comas).
SMTP_HOST	<code>smtp.gmail.com</code>	Host del servidor SMTP para el envío de correos.
SMTP_PORT	<code>587</code>	Puerto SMTP.
SMTP_USER	<code>gea.notificaciones@unilibrecucuta.edu.co</code>	Usuario/cuenta del servidor de correo.
SMTP_PASS	<code>(vacío)</code>	Contraseña SMTP (debe definirse para que el envío de correo funcione).
NOTIF_RECIPIENTS	<code>(vacío)</code>	Destinatarios de notificaciones de eventos universitarios (<code>university-recipients</code>).
PUBLISHED_RECIPIENTS	<code>(vacío)</code>	Destinatarios de notificaciones de contenido publicado (<code>published-recipients</code>).

Variable	Valor por defecto	Descripción
FRONTEND_URL	http://localhost:5173	URL base del frontend, usada para componer enlaces en los correos (<code>app.frontend-base-url</code>).
UPLOAD_DIR	uploads/	Directorio de almacenamiento de archivos subidos (<code>file.upload-dir</code>).
NOTIF_POOL_CORE	3	<i>Core pool size</i> del executor asíncrono de notificaciones.
NOTIF_POOL_MAX	6	<i>Max pool size</i> del executor asíncrono de notificaciones.
NOTIF_QUEUE_CAPACITY	30	Capacidad de la cola del executor asíncrono de notificaciones.
MANAGEMENT_PORT	8084	Puerto del servidor de administración (Actuador).

Valores fijos relevantes (no parametrizables por variable, pero útiles como referencia): `server.port=8083` , `server.address=0.0.0.0` , `server.servlet.context-path=/api` , `jwt.expiration=86400000` (24 h en ms), `spring.servlet.multipart.max-file-size=10MB` , extensiones permitidas `jpg,jpeg,png,pdf` , *rate limit* 60 peticiones/minuto.

Frontend (`.env.local` / `.env.production`)

Variable	Valor (local)	Valor (producción)	Descripción
VITE_API_URL	http://localhost:8083	https://api.gea.tudominio.com	Host+puerto del backend. El cliente Axios le añade el sufijo <code>/api</code> . <i>Fallback</i> en código: <code>http://localhost:8083</code> .

App móvil (Flutter `--dart-define`)

Variable	Valor por defecto	Descripción
API_BASE_URL	http://172.46.12.57:8083/api	URL completa de la API (incluye <code>/api</code>). Se define con <code>--dart-define=API_BASE_URL=...</code> al ejecutar o compilar. En dispositivo físico usar la IP de la PC; en producción, la URL pública del backend.

2. Backend (Spring Boot)

El backend de GEA es una aplicación **Spring Boot 3.4.3 sobre Java 21** que expone la API REST de la plataforma bajo el *context-path* `/api` (puerto 8083). Toda la lógica de negocio, la persistencia y la seguridad residen aquí; tanto el panel web (React) como la app móvil (Flutter) son únicamente clientes de esta API.

El paquete raíz del código es `com.calendario.callapp.callapp_backend`. A partir de aquí, todas las clases mencionadas se ubican en subpaquetes de esa raíz.

2.1 Arquitectura en capas

El backend sigue una **arquitectura en capas** clásica de tipo *Controller* → *Service* → *Repository* → *Entity*, con un flujo de datos basado en DTOs (objetos de transferencia) que aíslan el modelo de dominio (entidades JPA) de la API pública. Cada capa tiene una responsabilidad única y depende únicamente de la capa inmediatamente inferior:

- **Capa de presentación (`controller/`)**: clases anotadas con `@RestController`. Reciben las peticiones HTTP, validan el cuerpo entrante (`@Valid` sobre un *DTO Request*), delegan en la capa de servicio y devuelven la respuesta envuelta en `ApiResponse<T>` (ver `util/ApiResponse`). No contienen lógica de negocio. Ejemplo: `AuthController` expone `POST /auth/login` y se limita a invocar `authService.login(request)`.
- **Capa de servicio (`service/` + `service/impl/`)**: cada caso de uso se define como una **interfaz** en `service/` (p. ej. `AuthService`) y se implementa en `service/impl/` (p. ej. `AuthServiceImpl`). Aquí vive la lógica de negocio, la gestión transaccional (`@Transactional`) y la orquestación de repositorios, mappers y servicios de seguridad. Esta separación interfaz/implementación favorece la inversión de dependencias (principio D de SOLID): los controladores dependen de la abstracción, no de la implementación concreta.
- **Capa de acceso a datos (`repository/`)**: interfaces que extienden Spring Data JPA. Encapsulan el acceso a la base de datos mediante *query methods* derivados del nombre (p. ej. `findByNombreIgnoreCase`) y consultas optimizadas con `@Query` (p. ej. `getByCorreoOptimized`, que precarga rol y oficina para evitar el problema N+1 en el login).
- **Capa de dominio / persistencia (`entity/`)**: clases anotadas con `@Entity` que representan las tablas de la base de datos MySQL. Son el modelo persistente y nunca se exponen directamente al cliente.
- **Mappers (`mapper/`)**: traducen entre entidades y DTOs (entidad → *DTO Response* para la salida, *DTO Request* → entidad para la entrada), evitando filtrar campos sensibles (como el `password`) y desacoplando el contrato de la API del esquema de base de datos.

El flujo de una petición típica es: el cliente envía un **DTO Request** → el **Controller** lo valida y lo pasa al **Service** → el Service usa el **Repository** para leer/escribir **Entidades JPA** sobre **MySQL** → el Service emplea un **Mapper** para construir el **DTO Response** → el Controller devuelve ese DTO al cliente. La siguiente figura resume este recorrido.

 Figura: Capas del backend

Figura: Arquitectura en capas del backend

2.2 Estructura de paquetes

Bajo el paquete raíz `com.calendario.callapp.callapp_backend`, el código se organiza en los siguientes subpaquetes. El número de clases es orientativo del estado actual del repositorio.

Subpaquete	Contenido	Clases (aprox.)
<code>config/</code>	Configuración transversal de la aplicación: pools asíncronos, caché, codificador de contraseñas, auditoría JPA, propiedades de seguridad y <i>seeders</i> de datos iniciales.	8 + <code>package-info</code>
<code>controller/</code>	Controladores REST (<code>@RestController</code>). Punto de entrada HTTP de la API.	13
<code>dto/request/</code>	Objetos de entrada (cuerpo de las peticiones). Incluyen validaciones Bean Validation (<code>@NotBlank</code> , <code>@Email</code> , etc.).	18
<code>dto/response/</code>	Objetos de salida que se serializan a JSON. Construidos normalmente con el patrón <i>builder</i> de Lombok.	18
<code>entity/</code>	Entidades JPA (<code>@Entity</code>) mapeadas a las tablas MySQL, más enums de dominio (estados, roles, proveedores de autenticación).	22
<code>exception/</code>	Manejo centralizado de errores: <code>GlobalExceptionHandler</code> (<code>@RestControllerAdvice</code>) que normaliza las respuestas de error.	1
<code>mapper/</code>	Conversores entidad ↔ DTO.	5
<code>repository/</code>	Interfaces Spring Data JPA para el acceso a datos.	15
<code>security/</code>	Configuración y componentes de seguridad: cadena de filtros, JWT, <i>rate limiting</i> , <i>entry points</i> de error.	7 + <code>package-info</code>
<code>service/</code>	Interfaces de los casos de uso (contratos de la capa de negocio).	5
<code>service/impl/</code>	Implementaciones concretas de los servicios.	18

Subpaquete	Contenido	Clases (aprox.)
util/	Utilidades transversales: <code>ApiResponse<T></code> (envoltorio uniforme de respuestas) y <code>AuditRevisionListener</code> (auditoría con Hibernate Envers).	2

En la raíz del paquete se encuentra además la clase principal anotada con `@SpringBootApplication`, punto de arranque de la aplicación.

2.3 Capa de configuración

El paquete `config/` agrupa la configuración programática (clases `@Configuration`) y los inicializadores de datos. La siguiente tabla resume la responsabilidad real de cada clase; las más relevantes se detallan en subsecciones posteriores.

Clase	Anotaciones clave	Responsabilidad
<code>AsyncConfig</code>	<code>@Configuration</code> , <code>@EnableAsync</code> , <code>AsyncConfigurer</code>	Define el pool de hilos para tareas <code>@Async</code> (notificaciones) y el manejador global de excepciones asíncronas.
<code>CacheConfig</code>	<code>@Configuration</code> , <code>@EnableCaching</code>	Configura el <code>CacheManager</code> con Caffeine: <code>cache userDetails</code> con TTL de 45 s.
<code>PasswordConfig</code>	<code>@Configuration</code>	Expone el bean <code>BCryptPasswordEncoder</code> para el hash de contraseñas.
<code>JpaAuditingConfig</code>	<code>@Configuration</code>	Proveedor <code>AuditorAware<String></code> que identifica al usuario autor de cada cambio para la auditoría JPA.
<code>SecurityProperties</code>	<code>@Configuration</code> , <code>@ConfigurationProperties("app.security")</code> , <code>@Data</code>	Mapea propiedades de seguridad (CORS y <i>rate limit</i>) desde <code>application.properties</code> .
<code>AppSecurityProperties</code>	<code>@Configuration</code> , <code>@ConfigurationProperties("app.security")</code>	Variante de propiedades de seguridad (origenes CORS y <i>rate limit</i>) con getters/setters explícitos.
<code>DataInitializer</code>	<code>@Component</code> , <code>CommandLineRunner</code> , <code>@Transactional</code>	Siembra los datos maestros y de prueba en cada arranque.
<code>OficinaDataSeeder</code>	<code>@Configuration</code> , <code>CommandLineRunner</code>	Inserta oficinas adicionales (<i>Comunicaciones</i> , <i>Integridad</i>) si no existen.

Nota sobre `SecurityProperties` / `AppSecurityProperties`: ambas clases declaran el prefijo `app.security` y modelan los mismos conceptos (origenes CORS permitidos y límite de peticiones por minuto, con `enabled=true` y `requestsPerMinute=10` por defecto). Coexisten como puntos de configuración tipados; los valores reales se inyectan en los filtros y en `SecurityConfig` vía `@Value`.

2.3.1 AsyncConfig — pool de hilos asíncrono

`AsyncConfig` habilita el procesamiento asíncrono (`@EnableAsync`) e implementa `AsyncConfigurer` para personalizar tanto el executor por defecto como el tratamiento de errores. El bean `notificacionesExecutor` es un `ThreadPoolTaskExecutor` parametrizado por propiedades (con valores por defecto entre paréntesis):

- `core-pool-size` (3): hilos mínimos siempre activos.
- `max-pool-size` (6): hilos máximos cuando la cola se llena.
- `queue-capacity` (30): tareas en espera antes de crear hilos adicionales.
- Prefijo de nombre de hilo `notif-async-` (facilita el rastreo en logs).
- Política de rechazo `CallerRunsPolicy`: si el pool y la cola están saturados, la tarea se ejecuta en el hilo que la invocó en lugar de descartarse (contrapresión natural, sin pérdida de notificaciones).

El manejador `getAsyncUncaughtExceptionHandler()` registra en el log cualquier excepción no capturada en métodos `@Async` con valor de retorno `void`, evitando que un fallo asíncrono pase desapercibido.

```
@Configuration
@EnableAsync
@Slf4j
public class AsyncConfig implements AsyncConfigurer {

    @Value("${app.async.notifications.core-pool-size:3}")
    private int corePoolSize;

    @Value("${app.async.notifications.max-pool-size:6}")
    private int maxPoolSize;

    @Value("${app.async.notifications.queue-capacity:30}")
    private int queueCapacity;
```

```

@Bean(name = "notificacionesExecutor")
public ThreadPoolTaskExecutor notificacionesExecutor() {
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
    executor.setCorePoolSize(corePoolSize);
    executor.setMaxPoolSize(maxPoolSize);
    executor.setQueueCapacity(queueCapacity);
    executor.setThreadNamePrefix("notif-async-");
    // Si el pool y la cola se saturan, la tarea corre en el hilo llamador
    executor.setRejectedExecutionHandler(new ThreadPoolExecutor.CallerRunsPolicy());
    return executor;
}

@Override
public Executor getAsyncExecutor() {
    return notificacionesExecutor();
}

@Override
public AsyncUncaughtExceptionHandler getAsyncUncaughtExceptionHandler() {
    return (ex, method, params) ->
        log.error("[ASYNC-ERROR] Excepción no capturada en método '{}': {}",
            method.getName(), ex.getMessage(), ex);
}
}

```

2.3.2 CacheConfig — caché Caffeine

`CacheConfig` activa la abstracción de caché de Spring (`@EnableCaching`) y registra un `CaffeineCacheManager` con una única caché llamada `userDetails` . Esta caché almacena los `UserDetails` cargados durante la autenticación para evitar una consulta a base de datos en cada petición:

- **TTL:** `expireAfterWrite(45, SECONDS)` . El comentario del código lo justifica: *"un usuariobaneado deja de autenticarse en máx. 45 s"* . Es decir, el TTL acota la ventana durante la cual un cambio de estado/rol del usuario aún no se ha propagado.
- **Tamaño máximo:** `maximumSize(500)` entradas (límite de memoria).

Esta caché es consumida por `CustomUserDetailsService` (ver 2.4.4) mediante `@Cacheable` .

2.3.3 PasswordConfig — codificador BCrypt

Expone un único bean `BCryptPasswordEncoder` (tipo `PasswordEncoder`). Es el algoritmo usado para *hashear* las contraseñas al crear usuarios (`DataInitializer.crearUsuario` invoca `passwordEncoder.encode(...)`) y para verificarlas en el login (`AuthServiceImpl` usa `passwordEncoder.matches(...)`). `BCrypt` incorpora *salt* automático y un factor de coste configurable, resistente a ataques de fuerza bruta.

2.3.4 JpaAuditingConfig — auditoría JPA

Provee el bean `AuditorAware<String> auditorProvider` , que Spring Data JPA usa para rellenar los campos de auditoría (autor de creación/modificación) de las entidades. La lógica:

- Lee el `Authentication` del `SecurityContextHolder` .
- Si no hay autenticación, no está autenticado, o el principal es `anonymousUser` , devuelve **"SISTEMA"** (cambios automáticos como los *seeders*).
- En caso contrario devuelve `authentication.getName()` (el correo del usuario autenticado).

De este modo, cada registro queda atribuido a un autor identificable, ya sea un usuario real o el propio sistema.

2.3.5 DataInitializer — siembra de datos maestros y de prueba

`DataInitializer` implementa `CommandLineRunner` , por lo que su método `run` se ejecuta una vez completado el arranque, dentro de una transacción (`@Transactional`). Garantiza un estado funcional mínimo del sistema. Las operaciones, en orden:

1. **Roles base** (sólo si la tabla está vacía): `SuperAdmin` , `Comunicaciones` , `Oficina` , `Usuario Autenticado` .
2. **Oficinas** (sincronización idempotente): inserta/actualiza 11 oficinas oficiales (Facultad Derecho, Facultad CEAC, Facultad Ingeniería, Rectoría, Biblioteca, Consultorio Jurídico, Bienestar Universitario, Proyección Social, Marketing y Comunicaciones, Sistemas, Egresados) y **desactiva** las que no figuran en la lista oficial.
3. **Tipos de evento** (catálogo con color): Académico, Cultural, Deportivo, Institucional, Investigación, Bienestar, cada uno con su `colorHex` .
4. **Lugares físicos** (sincronización idempotente): 13 espacios con capacidad (Aula Máxima, Sala de Audiencias, Teatro, Biblioteca, Plaza de Banderas, etc.); también desactiva los obsoletos.
5. **Usuarios del sistema** (sólo si la tabla está vacía): tres cuentas base con contraseña `1234` (codificada con `BCrypt`) y proveedor `LOCAL` :
 - `Cesar@gmail.com` → rol **SuperAdmin** (oficina Sistemas).
 - `Comunicaciones@gmail.com` → rol **Comunicaciones** (oficina Marketing y Comunicaciones).
 - `oficina@gmail.com` → rol **Oficina** (oficina Rectoría).
 - Además vincula un avatar por defecto a los usuarios sin foto, copiándolo desde `classpath:static/assets/img/default-avatar.png` al directorio `uploads/` y registrándolo como `ArchivoAdjunto` público.

6. **Datos de prueba** (sólo si no hay solicitudes de evento): crea una `SolicitudEvento` de ejemplo (estado `APROBADA`) y una `SolicitudAnuncio` de ejemplo (estado `PUBLICADA`), con sus piezas gráficas.

Las credenciales y datos de prueba anteriores son **valores de desarrollo**. En un despliegue productivo deben cambiarse las contraseñas por defecto.

2.3.6 OficinaDataSeeder — oficinas complementarias

Componente `CommandLineRunner` independiente que inserta dos oficinas adicionales si aún no existen (`findByNombreIgnoreCase` vacío): `Comunicaciones` y `Integridad`. Es idempotente y complementa el catálogo de oficinas sembrado por `DataInitializer`.

2.4 Capa de seguridad

El paquete `security/` concentra toda la configuración de Spring Security. El modelo es **stateless basado en JWT**: no hay sesiones de servidor; cada petición se autentica por sí misma mediante un *Bearer token*. La cadena de filtros se complementa con un limitador de peticiones por IP y con manejadores de error que devuelven JSON uniforme.

2.4.1 SecurityConfig

`SecurityConfig` (`@EnableWebSecurity`, `@EnableMethodSecurity`) define el bean `SecurityFilterChain` con las siguientes características:

- **CORS**: habilitado mediante `corsConfigurationSource()`. Los orígenes permitidos se inyectan con `@Value("${app.security.cors.allowed-origins:*}")` y se aplican como *origin patterns*. Métodos permitidos: `GET`, `POST`, `PUT`, `DELETE`, `OPTIONS`, `PATCH`. Cabeceras permitidas: `Authorization`, `Cache-Control`, `Content-Type`, `Bypass-Tunnel-Reminder`. `allowCredentials = true`.
- **CSRF: deshabilitado** (`csrf.disable()`), apropiado para una API stateless consumida por clientes que envían el token explícitamente.
- **Gestión de sesión**: `SessionCreationPolicy.STATELESS` (no se crea ni usa `HttpSession`).
- **Cabeceras de seguridad**:
 - `frameOptions().deny()` — previene *clickjacking*.
 - `X-XSS-Protection: 1; mode=block`.
 - **Content-Security-Policy**: `default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data;;`.
- **Manejo de excepciones**: `authenticationEntryPoint` → `JwtAuthenticationEntryPoint` (401) y `accessDeniedHandler` → `JwtAccessDeniedHandler` (403).
- **Beans adicionales**: `AuthenticationManager` (obtenido de `AuthenticationConfiguration`) y `CorsConfigurationSource`.

Mapa de autorización ruta → roles (extraído literalmente de `authorizeHttpRequests`). El orden importa: Spring evalúa los *matchers* de arriba abajo y aplica el primero que coincide.

#	Método	Patrón de ruta	Regla de acceso
1	(todos)	<code>/auth/**</code> , <code>/api/auth/**</code>	<code>permitAll()</code> — público (login, auth Microsoft)
2	GET	<code>/app/eventos/publicados/**</code> , <code>/app/anuncios/publicados/**</code> , <code>/archivos/public/**</code> , <code>/app/eventos/agenda/export/pdf</code>	<code>permitAll()</code> — contenido público de la app móvil
3	(todos)	<code>/lugares-fisicos/**</code>	<code>authenticated()</code> — cualquier usuario autenticado
4	(todos)	<code>/admin/**</code>	<code>hasAnyRole("SUPER_ADMIN", "ADMIN")</code>
5	(todos)	<code>/comunicaciones/archivos/upload</code>	<code>hasAnyRole("SUPER_ADMIN", "ADMIN", "COMUNICACIONES", "OFICINA", "USUARIO_AUTENTICADO_APP")</code>
6	(todos)	<code>/comunicaciones/**</code>	<code>hasAnyRole("SUPER_ADMIN", "ADMIN", "COMUNICACIONES")</code>
7	(todos)	<code>/oficina/**</code>	<code>hasAnyRole("SUPER_ADMIN", "ADMIN", "COMUNICACIONES", "OFICINA")</code>
8	(todos)	<code>/app/solicitudes-anuncio/**</code>	<code>hasAnyRole("USUARIO_AUTENTICADO_APP", "SUPER_ADMIN", "ADMIN", "COMUNICACIONES", "OFICINA", "USUARIO_APP")</code>
9	(todos)	<code>/reportes/**</code>	<code>authenticated()</code>
10	(todos)	<code>/usuario/**</code>	<code>hasAnyRole("SUPER_ADMIN", "ADMIN", "COMUNICACIONES", "OFICINA", "USUARIO_APP", "USUARIO_AUTENTICADO_APP")</code>
11	(todos)	<code>anyRequest()</code>	<code>authenticated()</code> — por defecto, todo lo demás requiere autenticación

Spring Security antepone automáticamente el prefijo `ROLE_` al comparar con `hasAnyRole(...)`. Por eso `JwtAuthenticationFilter` y `CustomUserDetailsService` construyen autoridades con el prefijo explícito `ROLE_` (ver 2.4.3 y 2.4.4). El nombre concreto del rol proviene de `usuario.getRol().getSecurityRole()`.

Orden de los filtros (ambos registrados *antes* de `UsernamePasswordAuthenticationFilter`):

1. `RateLimiterFilter` — limita peticiones por IP antes de cualquier procesamiento de autenticación.
2. `JwtAuthenticationFilter` — valida el JWT y establece el `SecurityContext`.

2.4.2 JwtService

`JwtService` encapsula la emisión y validación de tokens JWT con la librería **jjwt** (firma HMAC-SHA). Lee dos propiedades: `jwt.secret` y `jwt.expiration`.

Fail-fast del secret (`@PostConstruct validarConfiguracion`): en el arranque valida la configuración y aborta si es insegura, lanzando `IllegalStateException`:

- Si `jwt.secret` es nulo o vacío.
- Si tiene menos de **32 caracteres** (requisito de longitud de clave HMAC).
- Si empieza por `dev_only_` y el entorno es de producción (detectado por los perfiles activos de Spring: contiene `prod`). Esto impide arrancar en producción con el secreto de desarrollo por defecto.

`generarToken(Usuario)` construye un JWT con:

- **subject** = correo del usuario.
- **claim id** = identificador del usuario.
- **claim rol** = `usuario.getRol().getSecurityRole()` (el rol de seguridad).
- **issuedAt** = ahora; **expiration** = ahora + `tokenExpiration`.
- Firmado con `getSigningKey()` (clave HMAC derivada del secreto en UTF-8).

```
public String generarToken(Usuario usuario) {
    return Jwts.builder()
        .setSubject(usuario.getCorreo())           // subject = correo
        .claim("id", usuario.getId())              // claim id
        .claim("rol", usuario.getRol().getSecurityRole()) // claim rol
        .setIssuedAt(new Date())
        .setExpiration(new Date(System.currentTimeMillis() + tokenExpiration))
        .signWith(getSigningKey())                 // firma HMAC
        .compact();
}
```

Métodos de lectura: `extractUsername` (subject), `extractRol` (claim rol) e `isTokenValid` (comprueba que la expiración sea futura). La extracción de claims (`extractClaim`) está envuelta en `try/catch` y devuelve `null` ante cualquier token inválido o manipulado, en lugar de propagar excepciones.

El siguiente diagrama muestra la secuencia completa de un login exitoso, desde la petición del cliente hasta la emisión del token por `JwtService`.

 Figura: Secuencia de autenticación (login)

Figura: Flujo de autenticación con JWT

En el código real, `AuthServiceImpl.login` además valida que el usuario exista y que su estado sea `ACTIVO` antes de comprobar la contraseña, y `AuthController` envuelve el `AuthResponse` resultante en `ApiResponse.success(...)` con el mensaje "Sesión iniciada correctamente".

2.4.3 JwtAuthenticationFilter

Filtro `OncePerRequestFilter` que se ejecuta una vez por petición. Su lógica:

1. Lee la cabecera `Authorization`. Si falta o no empieza por `"Bearer "`, deja pasar la petición sin autenticar (la autorización posterior decidirá si es pública o no).
2. Extrae el token (subcadena tras `"Bearer "`) y obtiene `correo` (`extractUsername`) y `rol` (`extractRol`).
3. Si `correo` y `rol` no son nulos, el token es válido (`isTokenValid`) y aún no hay autenticación en el contexto:
 - Carga los `UserDetails` con `userDetailsService.loadUserByUsername(correo)` (consulta cacheada — ver 2.4.4), lo que **revalida el estado del usuario contra la base de datos**.
 - Si el usuario está habilitado (`isEnabled()`), crea un `UsernamePasswordAuthenticationToken` con la autoridad `ROLE_<rol>` y lo fija en el `SecurityContextHolder`.
4. Cualquier excepción (usuario inexistente o deshabilitado) se registra como *warn* y limpia el contexto, dejando la petición como no autenticada.

Este doble control (firma del token + estado actual del usuario) permite que un usuario desactivado deje de autenticarse aunque su JWT siga vigente, dentro de la ventana del TTL de la caché (45 s).

2.4.4 CustomUserDetailsService

Implementa `UserDetailsService`. Su método `loadUserByUsername` está anotado con `@Cacheable(value = "userDetails", key = "#correo")`, de modo que el primer acceso consulta la base de datos (`getByCorreoOptimized`, que precarga rol y oficina) y los siguientes reutilizan el resultado cacheado durante 45 s.

Construye un `UserDetails` con:

- `username` = correo; `password` = hash almacenado.

- `disabled(!"ACTIVO".equalsIgnoreCase(estado))` : el usuario queda habilitado únicamente si su estado es `ACTIVO`; cualquier otro estado lo marca como deshabilitado (`enabled = false`).
- Autoridad `ROLE_<securityRole>` derivada de `usuario.getRol().getSecurityRole()`.

El método `invalidarCacheUsuario(String correo)` está anotado con `@CacheEvict(value = "userDetails", key = "#correo")`. Debe invocarse cuando cambian el estado o el rol de un usuario para **invalidar inmediatamente** su entrada cacheada, sin esperar al TTL.

2.4.5 RateLimiterFilter

Filtro `OncePerRequestFilter` que limita la tasa de peticiones **por IP de cliente**, mitigando fuerza bruta y abuso. Características:

- Se activa según `app.security.rate-limit.enabled` (por defecto `true`); si está deshabilitado, deja pasar todo.
- **Exclusiones:** las rutas `/archivos/public/`, `/swagger` y `/v3/api-docs` se omiten por rendimiento.
- **Ventana temporal:** resolución por **minuto** (`System.currentTimeMillis() / 60000`). El contador por IP se reinicia al cambiar de minuto.
- **Almacenamiento:** `ConcurrentHashMap<String, UserRequests>` actualizado atómicamente con `compute` y un `AtomicInteger` por entrada (seguro para concurrencia).
- **Límites diferenciados:** para rutas de autenticación (`/auth/` o `/api/auth/`) el máximo es `requests-per-minute` (por defecto **10/min**); para el resto, **5x** ese valor (50/min).
- Al superar el límite responde **HTTP 429 (Too Many Requests)** con un JSON `{"success":false, "message":"Has superado el limite de peticiones..."}`.
- **Identificación de IP** (`getClientIp`): da prioridad a la cabecera `X-Forwarded-For` (primer valor de la lista), validándola con una expresión regular de caracteres IPv4/IPv6 y longitud ≤ 45 para evitar *spoofing* por cabeceras malformadas; si no es válida, usa `request.getRemoteAddr()`.
- **Prevención de fugas de memoria:** el método `cleanupOldEntries`, anotado con `@Scheduled(fixedDelay = 600000)` (cada 10 min), elimina las entradas de minutos anteriores para que el mapa no crezca indefinidamente.

2.4.6 Entry points de error (401 / 403)

Dos componentes producen respuestas de error de seguridad uniformes en JSON (con `timestamp`, `status`, `error`, `message` y `path`), serializadas con `ObjectMapper`:

- `JwtAuthenticationEntryPoint` (`AuthenticationEntryPoint`): se invoca cuando un recurso protegido se solicita **sin autenticación válida**. Devuelve **HTTP 401 UNAUTHORIZED** con el mensaje *"Debes autenticarte para acceder a este recurso"*.
- `JwtAccessDeniedHandler` (`AccessDeniedHandler`): se invoca cuando un usuario **autenticado** intenta acceder a un recurso para el que **no tiene permisos**. Devuelve **HTTP 403 FORBIDDEN** con el mensaje *"No tienes permisos para acceder a este recurso"*.

Ambos se registran en `SecurityConfig` dentro de `exceptionHandling`, garantizando que los errores de autenticación y autorización tengan el mismo formato que el resto de la API.

2.5 Modelo de datos

El modelo de datos del backend se implementa mediante **JPA / Hibernate** sobre una base de datos **MySQL**. Cada tabla se representa con una clase `@Entity` del paquete `entity/`, cuyos identificadores se generan con estrategia `IDENTITY` (auto-incremento de MySQL). El modelo combina entidades de negocio (usuarios, solicitudes, publicaciones), entidades de catálogo (roles, tipos de evento, lugares físicos) y entidades de soporte operativo (dispositivos, reportes, notificaciones, archivos).

La mayoría de las entidades de negocio heredan de la superclase `BaseEntity`, que aporta de forma transparente los campos de **auditoría de creación y modificación** mediante el `listener` de Spring Data JPA. Además, las entidades relevantes para la trazabilidad están anotadas con `@Audited` (**Hibernate Envers**), lo que genera automáticamente tablas de historial (`_aud`) y registra cada cambio asociado a una revisión. La información de cada revisión (quién y desde qué IP) se almacena en la entidad personalizada `AuditRevisionEntity`.

 Figura: Modelo entidad-relación

Figura: Modelo entidad-relación del backend

2.5.1 Entidad base y auditoría

`BaseEntity` es una superclase abstracta anotada con `@MappedSuperclass` y `@EntityListeners(AuditingEntityListener.class)`. No genera tabla propia: sus columnas se materializan en cada tabla de las entidades que la extienden. Aporta cuatro campos de auditoría que Spring Data rellena automáticamente:

Campo	Columna	Anotación	Descripción
<code>fechaCreacion</code>	<code>fecha_creacion</code>	<code>@CreatedDate</code>	Fecha y hora de creación. <code>nullable = false</code> , <code>updatable = false</code> .
<code>fechaActualizacion</code>	<code>fecha_actualizacion</code>	<code>@LastModifiedDate</code>	Fecha y hora de la última modificación.
<code>usuarioCreacion</code>	<code>usuario_creacion</code>	<code>@CreatedBy</code>	Usuario que creó el registro. <code>updatable = false</code> .
<code>usuarioActualizacion</code>	<code>usuario_actualizacion</code>	<code>@LastModifiedBy</code>	Usuario de la última modificación.

Extienden `BaseEntity`: `Usuario`, `Oficina`, `DispositivoUsuario`, `SolicitudEvento`, `SolicitudAnuncio` y `LugarFisico`.

Auditoría histórica con Hibernate Envers. Las entidades anotadas con `@Audited` generan, por cada tabla, una tabla espejo de historial con sufijo `_aud` que conserva todas las versiones de cada fila junto al número de revisión. Están auditadas: `Usuario`, `RolEntity`, `Oficina`, `SolicitudEvento`, `SolicitudEventoParticipante`, `PublicacionEvento`, `SolicitudAnuncio`, `PublicacionAnuncio`, `TipoEventoCatalogo` y `LugarFisico`.

La entidad `AuditRevisionEntity` (tabla `audit_revision_info`) personaliza la entidad de revisión de Envers. Extiende `DefaultRevisionEntity` (que aporta `rev` y `revtstamp` mediante `@AttributeOverrides`) y añade contexto de seguridad mediante el `listener` `AuditRevisionListener`:

Campo	Columna	Descripción
id (heredado)	rev	Identificador de la revisión.
timestamp (heredado)	revtstamp	Marca temporal de la revisión.
nombreUsuario	nombre_usuario	Usuario que originó el cambio.
ipAddress	ip_address	Dirección IP desde la que se realizó el cambio.

Se eligió deliberadamente el nombre de tabla `audit_revision_info` (en lugar del `revinfo` por defecto) para evitar conflictos con esquemas antiguos y garantizar portabilidad.

2.5.2 Entidades principales

A continuación se documentan las **16 entidades** persistentes (no enumeraciones), agrupadas por dominio funcional.

Usuarios y roles

Usuario — tabla `usuarios` (extiende `BaseEntity`, `@Audited`). Representa a las personas que acceden a la plataforma.

Campo	Columna	Detalle
id	id_usuario	PK auto-incremental.
nombre	nombre	No nulo.
correo	correo	No nulo, único (índice <code>idx_usuario_correo</code>).
telefono	telefono	Longitud 30.
password	password	Hash de contraseña, no nulo.
rolEntity	id_rol (FK)	<code>@ManyToOne</code> → <code>RolEntity</code> .
oficina	id_oficina (FK)	<code>@ManyToOne</code> → <code>Oficina</code> (opcional).
estado	estado	Estado textual (p. ej. <code>ACTIVO</code>).
authProvider	auth_provider	Enum <code>AuthProvider</code> (<code>@Enumerated(STRING)</code>), no nulo.
microsoftOid	microsoft_oid	OID de Microsoft Entra, único (índice <code>idx_usuario_microsoft_oid</code>).
fotoUrl	foto_url	URL de avatar.

El método `@Transient getRol()` deriva el enum `Rol` a partir del nombre de `RolEntity`, manteniendo compatibilidad con la capa de seguridad sin duplicar datos.

RolEntity — tabla `roles` (`@Audited`). Catálogo de roles. Campos: `id` (`id_rol`, PK), `nombre` (`nombre`, no nulo, único, longitud 50) y `fechaCreacion`. *Nota:* a diferencia de las demás entidades de negocio, `RolEntity` no extiende `BaseEntity`.

Oficina — tabla `oficinas` (extiende `BaseEntity`, `@Audited`). Unidad organizativa que agrupa usuarios y solicitudes. Campos: `id` (`id_oficina`, PK), `nombre` (no nulo, único, longitud 120), `programaAcademico` (longitud 120), `descripcion` (longitud 500) y `activa` (booleano, no nulo).

DispositivoUsuario — tabla `dispositivos_usuario` (extiende `BaseEntity`). Registra los tokens de notificación push de cada usuario. Campos: `id` (`id_dispositivo`, PK), `token` (no nulo, único, longitud 500, índice `idx_dispositivo_token`), `usuario` (`id_usuario`, FK `@ManyToOne` no nulo → `Usuario`) y `fechaRegistro` (no nulo).

Eventos

SolicitudEvento — tabla `solicitudes_evento` (extiende `BaseEntity`, `@Audited`). Entidad central del flujo de eventos; concentra la solicitud y sus metadatos. Índices sobre `estado`, `id_oficina`, `fecha_evento` e `id_grupo_recurrencia`.

Campo	Columna	Detalle
id	id_solicitud	PK.
nombreEvento	nombre_evento	No nulo, longitud 160.

Campo	Columna	Detalle
descripcionEvento	descripcion_evento	Longitud 2000.
fechaEvento	fecha_evento	LocalDate , no nulo.
horaInicio / horaFin	hora_inicio / hora_fin	LocalTime , no nulos.
lugaresFisicos	tabla puente solicitud_evento_lugares	@ManyToMany → LugarFisico .
linkConexion	link_conexion	URL de transmisión.
responsableEvento	responsable_evento	Longitud 120.
tipoEventoCatalogo	id_tipo_evento (FK)	@ManyToOne → TipoEventoCatalogo .
estado	estado	Enum EstadoSolicitud , no nulo.
motivoRechazo	motivoRechazo	Longitud 1000.
fechaRevision	fecha_revision	Momento de aprobación/rechazo.
oficina	id_oficina (FK)	@ManyToOne no nulo → Oficina .
usuarioSolicitante	id_usuario_solicitante (FK)	@ManyToOne no nulo → Usuario .
usuarioRevisor	id_usuario_revisor (FK)	@ManyToOne opcional → Usuario .
piezaGraficaUrl	pieza_grafica_url	Imagen del evento.
requiereTransmision , requiereCubrimiento , esImportante , requierePiezaGrafica , esPrincipal	—	Banderas booleanas no nulas.
observaciones	observaciones	Longitud 2000.
frecuenciaRecurrencia	frecuencia_recurrencia	Enum FrecuenciaRecurrencia (por defecto NINGUNA).
fechaFinRecurrencia	fecha_fin_recurrencia	Fin de la serie recurrente.
idGrupoRecurrencia	id_grupo_recurrencia	Agrupar instancias de una misma serie.
participantes	—	@OneToMany → SolicitudEventoParticipante (cascade = ALL , orphanRemoval).

SolicitudEventoParticipante — tabla `solicitud_evento_participantes` (@Audited). Personas vinculadas a un evento. Campos: `id` (`id_participante`, PK), `nombre` (no nulo), `cargo`, `descripcion`, `fotoUrl`, `telefono`, `correo`, `tipo` (enum `TipoParticipante`, no nulo) y `solicitudEvento` (`id_solicitud_evento`, FK @ManyToOne no nulo → `SolicitudEvento`).

PublicacionEvento — tabla `publicaciones_evento` (@Audited). Versión publicada/visible de una solicitud de evento aprobada. Índices sobre `visible` e `id_solicitud_evento`. Campos: `id` (`id_publicacion_evento`, PK), `solicitudEvento` (`id_solicitud_evento`, FK @ManyToOne no nulo → `SolicitudEvento`), `tituloVisible` (no nulo), `descripcionVisible`, `piezaGraficaUrl`, `fechaPublicacion` (no nulo), `visible` (booleano, no nulo) y `usuarioPublicador` (`id_usuario_publicador`, FK @ManyToOne no nulo → `Usuario`).

TipoEventoCatalogo — tabla `tipos_evento` (@Audited). Catálogo de tipos de evento. Campos: `id` (`id_tipo_evento`, PK), `nombre` (no nulo, único, longitud 80), `descripcion`, `colorHex` (color de visualización, no nulo, longitud 7) y `activo` (booleano, no nulo).

Anuncios

SolicitudAnuncio — tabla `solicitudes_anuncio` (extiende `BaseEntity`, @Audited). Solicitud para publicar un anuncio durante un rango de fechas.

Campo	Columna	Detalle
id	id_solicitud_anuncio	PK.
titulo	titulo	No nulo, longitud 160.
descripcion	descripcion	Longitud 2000.
categoria	categoria	Longitud 100.
lugaresFisicos	tabla puente solicitud_anuncio_lugares	@ManyToMany → LugarFisico (cascade PERSIST/MERGE).
correoContacto / responsableAnuncio	—	Datos de contacto.
fechaInicioPublicacion / fechaFinPublicacion	—	Rango de vigencia (LocalDate).
horaInicio / horaFin	—	LocalTime .
piezaGraficaUrl	pieza_grafica_url	Imagen del anuncio.
estado	estado	Enum EstadoSolicitud , no nulo.

Campo	Columna	Detalle
<code>motivoRechazo</code> , <code>fechaRevision</code>	—	Datos de revisión.
<code>usuarioSolicitante</code>	<code>id_usuario_solicitante</code> (FK)	@ManyToOne no nulo → Usuario.
<code>oficina</code>	<code>id_oficina</code> (FK)	@ManyToOne opcional → Oficina.
<code>usuarioRevisor</code>	<code>id_usuario_revisor</code> (FK)	@ManyToOne opcional → Usuario.
<code>requierePiezaGrafica</code>	<code>requiere_pieza_grafica</code>	Booleano no nulo.

PublicacionAnuncio — tabla `publicaciones_anuncio` (@Audited). Versión publicada de un anuncio aprobado. Campos: `id` (`id_publicacion_anuncio`, PK), `solicitudAnuncio` (`id_solicitud_anuncio`, FK @ManyToOne no nulo → `SolicitudAnuncio`), `tituloVisible` (no nulo), `descripcionVisible`, `piezaGraficaUrl`, `fechaPublicacion` (no nulo), `visible` (booleano, no nulo) y `usuarioPublicador` (`id_usuario_publicador`, FK @ManyToOne no nulo → `Usuario`).

Compartido

LugarFisico — tabla `lugares_fisicos` (extiende `BaseEntity`, @Audited). Catálogo de espacios físicos reutilizables, referenciados vía @ManyToMany tanto por eventos como por anuncios. Campos: `id` (`id_lugar_fisico`, PK), `nombre` (no nulo, único, longitud 120), `descripcion`, `capacidad` (entero) y `activo` (booleano, no nulo, por defecto `true`).

ArchivoAdjunto — tabla `archivos_adjuntos`. Metadatos de archivos subidos (no extiende `BaseEntity` ni está auditada). Campos: `id` (`id_archivo_adjunto`, PK), `nombreOriginal` (no nulo), `nombreAlmacenado` (no nulo, único), `tokenAcceso` (no nulo, único, para acceso por URL), `contentType` (no nulo), `tamano` (bytes, no nulo), `publico` (booleano, no nulo) y `fechaCreacion` (no nulo).

Reportes y notificaciones

ReporteGenerado — tabla `reportes_generados`. Registro de reportes producidos. Campos: `id` (`id_reporte_generado`, PK), `nombre` (no nulo, longitud 160), `descripcion`, `formato` (no nulo, longitud 20), `fechaDesde` / `fechaHasta` (rango, no nulos), `alcance` (no nulo, longitud 30), `fechaCreacion` (no nulo), `idOficina` (Long simple, sin FK), `idTipoEvento` (Long simple, sin FK) y `usuarioGenerador` (`id_usuario_generador`, FK @ManyToOne no nulo → `Usuario`).

NotificacionEnviada — tabla `notificaciones_enviadas`. Bitácora de notificaciones emitidas. Campos: `id` (`id_notificacion_enviada`, PK), `tipo` (no nulo, longitud 30), `destinatarios` (longitud 1000), `asunto` (no nulo, longitud 200), `fechaEnvio` (no nulo), `exito` (booleano, no nulo) y `detalleError` (longitud 1000, motivo del fallo si `exito` = `false`).

2.5.3 Enumeraciones

El modelo define **5 enumeraciones** que se persisten como cadena (@Enumerated(EnumType.STRING)):

EstadoSolicitud — estado del flujo de aprobación de solicitudes de evento y anuncio.

Valor	Significado
PENDIENTE	Solicitud creada, a la espera de revisión.
APROBADA	Aprobada por un revisor.
RECHAZADA	Rechazada (con <code>motivoRechazo</code>).
PUBLICADA	Aprobada y ya publicada/visible.

Rol — roles de seguridad de la aplicación. Incluye lógica de mapeo (`fromNombre`, `getSecurityRole`) y de comprobación (`esAdministradorGlobal`, `esComunicaciones`, `esOficina`).

Valor	Significado
SUPER_ADMIN	Administrador global con control total.
COMUNICACIONES	Equipo de comunicaciones (revisión y publicación).
OFICINA	Usuario de oficina que crea solicitudes.
USUARIO_APP	Usuario básico de la app móvil.
USUARIO_AUTENTICADO_APP	Usuario autenticado de la app con permisos de oficina.
ADMIN	Administrador (equivalente de seguridad a <code>SUPER_ADMIN</code>).
USUARIO	Rol por defecto / genérico.

FrecuenciaRecurrencia — periodicidad de los eventos recurrentes.

Valor	Significado
NINGUNA	Evento único (valor por defecto).

Valor	Significado
DIARIA	Se repite cada día.
SEMANAL	Se repite cada semana.
MENSUAL	Se repite cada mes.

TipoParticipante — rol de un participante dentro de un evento.

Valor	Significado
ORGANIZADOR	Responsable de organizar el evento.
INVITADO	Persona invitada / ponente.
PATROCINADOR_ALIADO	Patrocinador o aliado del evento.

AuthProvider — proveedor de autenticación del usuario.

Valor	Significado
LOCAL	Credenciales locales (correo y contraseña).
MICROSOFT	Autenticación federada vía Microsoft Entra ID.

2.6 Capa de repositorios

El paquete `repository/` contiene **14 interfaces** que extienden `JpaRepository<Entidad, Long>` de Spring Data JPA. Al heredar de `JpaRepository` cada repositorio dispone de inmediato del CRUD estándar (`save`, `saveAll`, `findById`, `findAll`, `delete`, `count`, etc.) y de la posibilidad de declarar *query methods* derivados del nombre del método (p. ej. `findByActivoTrueOrderByNombreAsc`, `existsByCorreo`, `countByOficinaId`), que Spring traduce automáticamente a SQL sin escribir consultas.

Para los casos que requieren control fino sobre el SQL/JPQL generado, los repositorios usan consultas explícitas con `@Query`. Dos patrones recorren toda la capa y merecen explicación:

- **JOIN FETCH contra el problema N+1.** Las asociaciones JPA son por defecto *lazy*: si se carga una lista de N solicitudes y luego, al serializarlas, se accede a su oficina, su tipo de evento y su solicitante, Hibernate ejecutaría una consulta adicional por cada asociación de cada fila (N+1 consultas). Para evitarlo, las consultas optimizadas (sufijo `Optimized` o nombre `getAllVisibleOptimized`, `getByCorreoOptimized`, etc.) precargan en **una sola sentencia** las asociaciones necesarias mediante `JOIN FETCH` (o `LEFT JOIN FETCH` cuando la relación es opcional). Así, el grafo de objetos llega completo a la capa de servicio y la serialización a DTO no dispara más consultas.
- **Paginación de seguridad (`Pageable` / `PageRequest`).** Varias consultas que podrían devolver tablas enteras aceptan un `Pageable`. En la capa de servicio se les pasa un `PageRequest.of(0, N)` con un tope explícito (p. ej. 200 para listados de revisión, 300 para publicaciones visibles, 50 para "próximos"). Esto acota la cantidad de filas materializadas y protege la memoria frente a tablas que crecen con el uso, aunque la API no exponga aún paginación al cliente.

La siguiente tabla resume los 14 repositorios y sus métodos personalizados más relevantes.

Repositorio	Entidad	Métodos personalizados destacados
<code>UsuarioRepository</code>	<code>Usuario</code>	<code>getByCorreoOptimized</code> (JOIN FETCH de rol + LEFT JOIN FETCH de oficina; base del login y del <code>UserDetailsService</code>); <code>existsByCorreo/Telefono/MicrosoftOid</code> ; <code>findByTelefonoExcluyendo</code> / <code>findByMicrosoftOidExcluyendo</code> (validan unicidad excluyendo el propio id en ediciones); <code>searchUsuarios(q, rolName, estado)</code> (búsqueda dinámica con filtros opcionales <code>:param IS NULL OR ...</code>); <code>countByOficinaId</code> .
<code>SolicitudEventoRepository</code>	<code>SolicitudEvento</code>	<code>findByIdOptimized</code> y <code>getAllByOficinaIdOptimized</code> / <code>getAllUniqueWithAssociations</code> (LEFT JOIN FETCH de oficina, tipo y solicitante); <code>findConflictsBulk</code> (detección de choques de lugar/horario); <code>findAllByIdGrupoRecurrencia</code> (instancias de una serie); familia de <code>count...Grouped</code> y <code>count...Filtered</code> para el dashboard y reportes; conjunto de <code>findBy...FechaCreacionBetween...</code> para reportes.
<code>PublicacionEventoRepository</code>	<code>PublicacionEvento</code>	<code>findProximos(hoy, Pageable)</code> y <code>findPublicadasEnRango(inicio, fin)</code> (JOIN FETCH + orden por importancia/fecha/hora); <code>getAllVisibleOptimized(Pageable)</code> ; <code>findByIdAndVisibleUnique</code> ; <code>findBySolicitudEventoId</code> / <code>findBySolicitudEventoIdIn</code> (carga masiva de visibilidad sin N+1).
<code>SolicitudAnuncioRepository</code>	<code>SolicitudAnuncio</code>	<code>findByIdOptimized</code> , <code>getAllByOficinaIdOptimized</code> , <code>getAllUniqueWithAssociations</code> (con <code>DISTINCT</code> por el JOIN FETCH de la colección de lugares); counts agrupados y filtrados análogos a eventos; consultas de reporte por oficina/categoria/estado/mes.

Repositorio	Entidad	Métodos personalizados destacados
PublicacionAnuncioRepository	PublicacionAnuncio	findBySolicitudAnuncioId / findBySolicitudAnuncioIdIn ; findByVisibleTrueOrderByFechaPublicacionDesc ; findByIdAndVisibleTrue .
SolicitudEventoParticipanteRepository	SolicitudEventoParticipante	findBySolicitudEventoId ; deleteBySolicitudEventoId (limpieza previa al repoblar participantes).
ReporteGeneradoRepository	ReporteGenerado	Sobrescribe findAll y findById con @Query + LEFT JOIN FETCH de usuario generador y su oficina; findByUsuarioGeneradorIdOrderByFechaCreacionDesc , findByUsuarioGeneradorOficinaIdOrderByFechaCreacionDesc , findByFechaCreacionBetween .
OficinaRepository	Oficina	findByNombreIgnoreCase (idempotencia de los seeders); findByActivaTrueOrderByNombreAsc .
LugarFisicoRepository	LugarFisico	findByActivoTrueOrderByNombreAsc ; findByNombreIgnoreCase .
TipoEventoCatalogoRepository	TipoEventoCatalogo	findByActivoTrueOrderByNombreAsc ; findByNombreIgnoreCaseAndActivoTrue (resolución del tipo al crear una solicitud); findByNombreIgnoreCase .
RolRepository	RolEntity	findByNombre (resolución de rol en seeders y alta de usuarios Microsoft).
DispositivoUsuarioRepository	DispositivoUsuario	findByToken ; findByUsuarioId ; deleteByToken (registro/baja de tokens FCM).
ArchivoAdjuntoRepository	ArchivoAdjunto	findByTokenAccesoAndPublicoTrue (descarga pública por token); findByNombreOriginal .
NotificacionEnviadaRepository	NotificacionEnviada	Sólo CRUD heredado; se usa como bitácora de envíos (escritura desde NotificacionServiceImpl).

2.6.1 Consultas optimizadas destacadas

Algunas consultas concentran lógica de negocio en el propio JPQL y conviene detallarlas:

- **UsuarioRepository.getByCorreoOptimized** — `SELECT u FROM Usuario u JOIN FETCH u.rolEntity LEFT JOIN FETCH u.oficina WHERE u.correo = :correo` . Carga el usuario con su rol (obligatorio) y su oficina (opcional) en una sola consulta. Es la base de **todo** la autenticación: la usan `AuthServiceImpl.login` , `CustomUserDetailsService` (cacheada) y la mayoría de servicios para resolver el usuario autenticado a partir de `authentication.getName()` .
- **PublicacionEventoRepository.findProximos / getAllVisibleOptimized** — ambas filtran por `p.visible = true` , hacen `JOIN FETCH` de la solicitud, su oficina y su tipo de evento, y ordenan por `s.esImportante DESC` , `fecha` , `hora` . `findProximos` añade `s.fechaEvento >= :hoy` para devolver sólo lo venidero; `getAllVisibleOptimized` ordena además por `fechaPublicacion DESC` . La paginación (`Pageable`) limita el resultado (50 próximos, 300 visibles).
- **SolicitudEventoRepository.findConflictsBulk** — corazón de la validación de agenda. Recibe una **lista** de fechas, una lista de ids de lugar, una franja horaria y un id a excluir. Devuelve las solicitudes `APROBADA` o `PUBLICADA` que comparten lugar y se **solapan en horario** (`s.horaInicio < :horaFin AND s.horaFin > :horaInicio`), opcionalmente excluyendo una solicitud concreta (`:id IS NULL OR s.id <> :id`). Al aceptar una lista de fechas, permite comprobar de un golpe **todo una serie recurrente** sin caer en N+1. La detección de solapamiento es estándar de intervalos: dos rangos se cruzan si cada uno empieza antes de que el otro termine.
- **Counts agrupados** (`countByEstadoGrouped` , `countByEstadoGroupedByOficina`) — devuelven `List<Object[]>` con `[estado, total]` , que los servicios convierten a `Map<EstadoSolicitud, Long>` . Evitan ejecutar una consulta `count` por cada estado, resolviendo el resumen del dashboard en una sola pasada agregada con `GROUP BY` .

2.7 Capa de servicios

El paquete `service/impl/` reúne **17 implementaciones** que contienen la lógica de negocio del backend. El patrón es uniforme: clase anotada con `@Service` , inyección de dependencias por constructor mediante `@RequiredArgsConstructor` de Lombok (campos `private final`) y gestión transaccional declarativa con `@Transactional` (en modo `readOnly = true` para las consultas, lo que permite a Hibernate optimizar y omitir el *dirty checking*). Los métodos de lectura/escritura reciben el `Authentication` de Spring Security cuando necesitan resolver al usuario actual, y señalan los errores de negocio con `ResponseStatusException` y el `HttpStatus` adecuado, que el `GlobalExceptionHandler` normaliza a JSON.

Algunos servicios implementan una **interfaz** del paquete `service/` (`AuthServiceImpl → AuthService` , `ArchivoServiceImpl → ArchivoStorageService` , `LugarFisicoServiceImpl → LugarFisicoService` , `DispositivoUsuarioServiceImpl → DispositivoUsuarioService`); el resto se exponen como clases concretas inyectadas directamente en los controladores.

2.7.1 Autenticación

AuthServiceImpl (implementa `AuthService`). Login local con credenciales. Su único método `login(AuthRequest)` (transacción de solo lectura):

1. Resuelve el usuario con `getByCorreoOptimized` ; si no existe responde **401** con el mensaje genérico *"Correo o contraseña incorrectos"* (no revela si el correo existe).

2. Valida que el estado sea `ACTIVO`; en caso contrario, **401** "Tu cuenta está inactiva".
3. Compara la contraseña con `passwordEncoder.matches(plano, hashAlmacenado)` (BCrypt); si no coincide, **401** genérico.
4. Genera el JWT con `jwtService.generarToken(usuario)` y construye un `AuthResponse` (token + nombre, correo, rol, id/nombre de oficina y `fotoUrl`).

Todos los intentos fallidos se registran con `log.warn` para trazabilidad. Dependencias: `UsuarioRepository`, `JwtService`, `PasswordEncoder`.

MicrosoftAuthServiceImpl — SSO con **Azure AD / Microsoft Entra ID**. Su método `autenticar(MicrosoftAuthRequest)`:

1. Exige que `microsoft.tenant-id` y `microsoft.client-id` estén configurados; si no, responde **501 Not Implemented**.
2. Valida el `idToken` recibido descargando las claves del `issuer` (`JwtDecoders.fromIssuerLocation("https://login.microsoftonline.com/<tenant>/v2.0")`) y decodificándolo; un token inválido produce **401**.
3. `validarAudiencia`: comprueba que el claim `aud` contenga el `client-id` propio (impide que un token emitido para otra app se reutilice aquí).
4. Extrae los claims `preferred_username` (correo), `name` y `oid`. **Aprovisionamiento automático (JIT)**: si el correo no existe, crea un usuario nuevo con estado `ACTIVO`, contraseña aleatoria (`UUID`), `authProvider = MICROSOFT` y rol "Usuario Autenticado"; si ya existe, valida que esté `ACTIVO` y actualiza su `microsoftOid` y proveedor.
5. Devuelve un `AuthResponse` con un JWT propio de GEA generado por `JwtService` (el token de Microsoft sólo sirve para autenticar; a partir de ahí el sistema usa su propio JWT).

Dependencias: `UsuarioRepository`, `RolRepository`, `JwtService`.

2.7.2 Gestión de eventos — `SolicitudEventoServiceImpl`

Es el **servicio más complejo del backend**. Orquesta el ciclo de vida completo de un evento, desde la solicitud hasta la publicación, incluyendo eventos recurrentes (series), detección de conflictos de lugares y notificaciones por correo y push. Inyecta 11 dependencias (repositorios de solicitud/publicación/usuario/oficina/tipo/participante/lugar/dispositivo, los dos mappers, `NotificacionServiceImpl` y `PushNotificationService`).

Ciclo de vida y transiciones de estado (`EstadoSolicitud`): toda solicitud nace `PENDIENTE`; un revisor la lleva a `APROBADA` o `RECHAZADA`; una solicitud aprobada se `PUBLICADA` (creando su `PublicacionEvento` visible). Las transiciones se resumen así:

Método	Transición	Reglas y efectos
<code>crear</code>	→ <code>PENDIENTE</code>	Valida horario, permisos de rol, resuelve oficina y tipo, comprueba conflictos, guarda participantes; si hay recurrencia genera la serie; notifica creación.
<code>aprobar</code>	<code>PENDIENTE/RECHAZADA</code> → <code>APROBADA</code>	Fija revisor y <code>fechaRevision</code> , limpia <code>motivoRechazo</code> ; notifica aprobación. Si es maestra de serie, delega en <code>aprobarSerie</code> .
<code>rechazar</code>	→ <code>RECHAZADA</code>	Guarda <code>motivoRechazo</code> , revisor y fecha; notifica rechazo. Serie → <code>rechazarSerie</code> .
<code>publicar</code>	<code>APROBADA/PUBLICADA</code> → <code>PUBLICADA</code>	Crea/actualiza la <code>PublicacionEvento</code> (visible), sincroniza la pieza gráfica, notifica por correo y dispara push si el evento es importante. Serie → <code>publicarSerie</code> .
<code>eliminarPublicacion</code>	<code>PUBLICADA</code> → <code>APROBADA</code>	Borra la publicación y devuelve la solicitud a aprobada (la despublica sin perderla).
<code>toggleVisibilidad</code>	(sin cambiar estado)	Marca la <code>PublicacionEvento</code> como visible/oculta.

Creación (`crear`). Valida que `horaFin > horaInicio` (`validarHorario`), que el rol pueda crear eventos (oficina o admin) y resuelve la oficina (un admin puede elegirla con `idOficina`; un usuario de oficina usa la suya). Mapea el request a la entidad, **comprueba conflictos** con `checkConflicts` antes de persistir y guarda participantes. Si el request trae `frecuenciaRecurrencia != NINGUNA` y `fechaFinRecurrencia`, marca la primera como maestra (`esPrincipal = true`), le asigna un `idGrupoRecurrencia` (`UUID`) y genera las instancias.

Recurrencia (`generarInstanciasRecurrentes`). Recorre las fechas según la frecuencia (`DIARIA` → `plusDays(1)`, `SEMANAL` → `plusWeeks(1)`, `MENSUAL` → `plusMonths(1)`) hasta `fechaFinRecurrencia`, con dos salvaguardas de auditoría: la fecha fin es **obligatoria** y el número de instancias se limita a **100** (`MAX_INSTANCIAS`, anti-DoS). Antes de clonar, precarga **de una sola vez** todos los posibles conflictos de la serie con `findConflictsBulk` y luego verifica en memoria fecha a fecha (evitando N+1); si una instancia choca de lugar y horario, aborta la serie con **409 Conflict**. Cada instancia se clona con `esPrincipal = false`, `frecuencia = NINGUNA` y el mismo `idGrupoRecurrencia`.

Conflictos de lugares (`checkConflicts`). Para una fecha, los lugares, una franja horaria y un id a excluir, llama a `findConflictsBulk`; si hay choque lanza **409** indicando el lugar y el evento en conflicto. Sólo se consideran solicitudes `APROBADA` o `PUBLICADA` (una solicitud pendiente no reserva el espacio).

Operaciones sobre series. `aprobarSerie`, `rechazarSerie`, `publicarSerie`, `eliminarSerie`, `actualizarSerie` y `toggleVisibilidadSerie` aplican la acción a todas las instancias de un `idGrupoRecurrencia` y notifican **una sola vez** (sobre la instancia maestra, con sufijo "(Serie Completa)" / "(Serie de eventos)"). Al editar la maestra, `propagarCambiosMaster` sincroniza datos y publicaciones de las instancias secundarias, cargando todas sus publicaciones de golpe con `findBySolicitudEventoIdIn` para no incurrir en N+1.

Notificaciones push de eventos importantes (`triggerImportantEventPush`). Tras publicar, si `solicitud.esImportante`, recopila todos los tokens FCM distintos y envía un `multicast` con `PushNotificationService`; los errores se capturan y registran sin interrumpir la publicación.

A continuación, un extracto anotado del método `publicar`, que materializa la transición `APROBADA` → `PUBLICADA`:

```
@Transactional
public PublicacionEventoResponse publicar(Long id, PublicacionEventoRequest request, Authentication authentication) {
    SolicitudEvento solicitud = buscarSolicitud(id);
```

```
// Si es la maestra de una serie, publicar toda la serie y devolver la publicación principal
if (solicitud.getEsPrincipal() && solicitud.getIdGrupoRecurrencia() != null) {
    publicarSerie(solicitud.getIdGrupoRecurrencia(), request, authentication);
    PublicacionEvento pubPrincipal = publicacionEventoRepository.findBySolicitudEventoId(id)
        .orElseThrow(() -> new RuntimeException(HttpStatus.INTERNAL_SERVER_ERROR, "Error al recuperar publicación maestra"));
    triggerImportantEventPush(pubPrincipal);
    return publicacionEventoMapper.toResponse(pubPrincipal);
}

// Sólo se publica lo que ya fue aprobado (o se reactualiza lo ya publicado)
if (solicitud.getEstado() != EstadoSolicitud.APROBADA && solicitud.getEstado() != EstadoSolicitud.PUBLICADA) {
    throw new RuntimeException(HttpStatus.BAD_REQUEST, "Solo puedes publicar una solicitud aprobada");
}

Usuario publicador = obtenerUsuario(authentication);
// Reutiliza la publicación si ya existía (re-publicación), o crea una nueva
PublicacionEvento publicacion = publicacionEventoRepository.findBySolicitudEventoId(id).orElseGet(PublicacionEvento::new);
publicacion.setSolicitudEvento(solicitud);
publicacion.setTituloVisible(request.getTituloVisible());
publicacion.setDescripcionVisible(request.getDescripcionVisible());
publicacion.setPiezaGraficaUrl(request.getPiezaGraficaUrl());
publicacion.setFechaPublicacion(request.getFechaPublicacion() != null ? request.getFechaPublicacion() : LocalDateTime.now());
publicacion.setVisible(true);
publicacion.setUsuarioPublicador(publicador);

solicitud.setEstado(EstadoSolicitud.PUBLICADA); // transición de estado
solicitudEventoRepository.save(solicitud);
PublicacionEvento guardada = publicacionEventoRepository.save(publicacion);

notificacionService.notificarPublicacionEvento(/* ... datos del evento y correo del solicitante ... */);
triggerImportantEventPush(guardada); // push FCM si es importante
return publicacionEventoMapper.toResponse(guardada);
}
```

2.7.3 Gestión de anuncios — `SolicitudAnuncioServiceImpl`

Gestiona el ciclo **solicitud** → **publicación** de anuncios, paralelo al de eventos pero **sin recurrencia ni conflictos de lugar** (un anuncio es vigente durante un rango de fechas, no reserva un espacio en una franja). Métodos principales: `crear` (estado `PENDIENTE`; si el solicitante es `USUARIO_AUTENTICADO_APP` la oficina queda nula), `actualizar` (sólo el autor o un admin), `aprobar`, `rechazar`, `publicar` (crea/actualiza la `PublicacionAnuncio` visible, rellenando título/descripción/pieza desde la solicitud si el request no los trae), `listarPublicados`, `eliminarSolicitud`, `eliminarPublicacion` (revierte a `APROBADA`), `toggleVisibilidad` y `updatePublicacion`. Cada transición delega en `NotificacionServiceImpl` (creación, aprobación, rechazo, publicación). Los listados se "enriquecen" con la visibilidad real consultando las publicaciones por lotes (`findBySolicitudAnuncioIdIn`) para evitar N+1.

2.7.4 Notificaciones

`NotificacionServiceImpl` — envío de correos **asíncronos**. Todos sus métodos `notificar...` están anotados con `@Async("notificacionesExecutor")`, de modo que se ejecutan en el pool de hilos dedicado (ver 2.3.1) y **no bloquean** la petición HTTP que los origina. Tipos de correo cubiertos:

Método	Disparador	Destinatarios
<code>notificarCreacionEvento</code> / <code>notificarCreacionAnuncio</code>	Nueva solicitud	Buzones de gestión (<code>university-recipients</code>).
<code>notificarAprobacionEvento</code> / <code>notificarAprobacionAnuncio</code>	Aprobación	Correo del solicitante.
<code>notificarRechazoEvento</code> / <code>notificarRechazoAnuncio</code>	Rechazo (con motivo)	Correo del solicitante.
<code>notificarPublicacionEvento</code> / <code>notificarPublicacionAnuncio</code>	Publicación	Solicitante + buzones de gestión + lista masiva (<code>published-recipients</code>), en BCC para proteger la privacidad.
<code>enviarCorreoPruebaEvento</code>	Endpoint de prueba	Destinatario indicado.

El método privado `enviarCorreo` construye un `MimeMessage` (HTML, UTF-8), respeta los interruptores de configuración (`email-enabled`, SMTP configurado, lista de destinatarios no vacía) y, cuando la pieza gráfica apunta a `/archivos/public/`, la incrusta *inline* en el correo (o la adjunta si no es imagen) resolviéndola con `ArchivoServiceImpl`. **Cada envío —exitoso o fallido— se registra en la bitácora** `NotificacionEnviada` (`registrarNotificacion`), incluyendo el motivo del fallo (`EMAIL_DESACTIVADO`, `SMTP_NO_CONFIGURADO`, `SIN_DESTINATARIOS` o el mensaje de la excepción). Dependencias: `JavaMailSender`, `NotificacionEnviadaRepository`, `PlantillaCorreoServiceImpl`, `ArchivoServiceImpl`.

`PlantillaCorreoServiceImpl` — renderizado de **plantillas HTML de correo**. Su método `render(nombrePlantilla, variables)` lee el HTML desde `classpath:mail-templates/<nombre>` y sustituye los marcadores `${clave}` por los valores del mapa. Para prevenir inyección de HTML en el correo, **escapa** cada valor con `HtmlUtils.htmlEscape` antes de insertarlo. Si la plantilla no existe, responde **500**.

PushNotificationService — notificaciones push vía **Firebase Cloud Messaging (FCM)**. En `@PostConstruct init()` inicializa el *Firebase Admin SDK* leyendo `firebase-service-account.json` del classpath; si el archivo no está, registra una advertencia y opera en **modo simulado** (las notificaciones sólo se escriben en el log, útil en desarrollo). `sendMulticastNotification(tokens, title, body, data)` construye un `MulticastMessage` (notificación + *data payload*) y lo envía con `sendEachForMulticast`, registrando los conteos de éxito/fallo. Los errores se capturan y loguean sin propagarse.

2.7.5 Reportes

ReporteServiceImpl — generación y exportación de reportes de gestión. Métodos públicos: `crearReporte` (persiste la definición del reporte: nombre, formato, rango de fechas, alcance, oficina y tipo de evento), `listarReportes` (filtrado por rol — un admin ve todos; una oficina ve los de su oficina o los propios), `obtenerDashboardStats` (estadísticas agregadas para los gráficos del panel, combinando los counts filtrados de eventos y anuncios por tipo, oficina, estado y mes), `generarResumen`, `exportarReporteGenerado` / `exportarPdf` / `exportarXlsx` y `actualizarReporte`. La **exportación a PDF** usa OpenPDF (`com.lowagie.text`) con la paleta de marca GEA (rojo `#CE1126`, grises) y un pie de página personalizado (`PdfPageEventHelper`); la **exportación a XLSX** usa Apache POI (`XSSFWorkbook`) con estilos de celda, cabeceras coloreadas y filas zebra. Valida el rango de fechas y normaliza el formato solicitado. Dependencias: los repositorios de solicitud de evento/anuncio, usuario y `ReporteGeneradoRepository`, más un `EntityManager`.

AgendaPdfService — exportación de la **agenda de eventos publicados a PDF** (endpoint público de la app móvil). `exportarAgendaPdf(desde, hasta)` valida el rango (no nulo, `hasta >= desde`, máximo **366 días**), obtiene las publicaciones visibles con `findPublicadasEnRango` (usando `plusDays(1)` para que `hasta` sea inclusivo), las **agrupa por día** conservando el orden de la consulta (importancia, fecha, hora) y compone un documento A4 con OpenPDF: encabezado con el rango, un bloque por día con sus eventos, o un mensaje de "sin eventos" si el rango está vacío, y un pie de página de marca (`PieDeAgenda`). Dependencia: `PublicacionEventoRepository`.

2.7.6 Archivos — ArchivoServiceImpl

Implementa `ArchivoStorageService`: **almacenamiento de archivos** en disco con metadatos en base de datos. En `@PostConstruct` resuelve y crea el directorio de subida (`file.upload-dir`, por defecto `uploads`). `guardar(MultipartFile)`:

1. **Valida** el archivo (`validarArchivo`): tamaño máximo (`file.max-size-bytes`, por defecto 10 MB), extensión permitida (`jpg, jpeg, png, pdf`) y *content-type* permitido (`image/jpeg`, `image/png`, `application/pdf`). Cualquier violación produce **400**.
2. Sanea el nombre original (`StringUtils.cleanPath`, contra *path traversal*), genera un nombre de almacenamiento único (`UUID` + extensión) y copia el contenido al directorio.
3. Persiste un `ArchivoAdjunto` con un `tokenAcceso` único (UUID sin guiones) y `publico = true`, y devuelve un `ArchivoResponse` con la URL pública `/archivos/public/<token>`.
4. **Transaccionalidad disco→BD**: si falla el guardado de metadatos, borra el archivo del disco para no dejar huérfanos.

`cargarPublicoComoRecurso(token)` valida el token con una expresión regular (`^[a-zA-Z0-9]{20,}$`), busca el metadato público, comprueba que la ruta resuelta no escape del directorio raíz (defensa *path traversal*) y devuelve el `Resource`; ante cualquier fallo entrega un **recurso por defecto** (avatar o imagen de evento) en lugar de propagar el error.

2.7.7 Administración

UsuarioServiceImpl — CRUD de usuarios del panel. `crearUsuario` valida datos básicos y la **unicidad** de correo, teléfono y `microsoft0id`, **encripta la contraseña** con `BCryptPasswordEncoder.encode`, resuelve rol y oficina y fija estado `ACTIVO`. `actualizarUsuario` aplica los cambios (re-encripta la contraseña sólo si llega una nueva) y, de forma crítica para la seguridad, invoca `customUserDetailsService.invalidateCacheUsuario(correo)` para **purgar la entrada cacheada** del usuario (`@CacheEvict`) en cuanto cambian su estado o rol, sin esperar al TTL de 45 s de la caché `userDetails` (ver 2.4.4). `resolveOficina` aplica reglas de negocio por rol (Comunicaciones exige la oficina "Comunicaciones"; `USUARIO_AUTENTICADO_APP` la oficina "Usuarios"). Para romper la dependencia circular con seguridad, `CustomUserDetailsService` se inyecta con `@Lazy`.

OficinaServiceImpl — `crear` (alta de oficina, `activa = true` por defecto) y `listar` (oficinas activas ordenadas por nombre).

LugarFisicoServiceImpl (implementa `LugarFisicoService`) — `listarActivos` (lugares activos ordenados, vía mapper) y `obtenerPorId` (404 si no existe).

TipoEventoServiceImpl — `listarActivos` y `crear` (alta de tipo de evento con color, `activo = true` por defecto).

DispositivoUsuarioServiceImpl (implementa `DispositivoUsuarioService`) — gestiona los **tokens FCM** del usuario autenticado. `registrarToken` hace *upsert*: si el token ya existe lo reasigna al usuario actual y refresca la fecha; si no, lo crea. `removerToken` elimina el token, validando primero que pertenezca al usuario que lo solicita (**403** en caso contrario).

2.7.8 Soporte

DashboardServiceImpl — métricas y resumen del panel. `resumen(Authentication)` distingue el **alcance** por rol: un administrador/Comunicaciones obtiene cifras **globales** (totales de solicitudes de evento y anuncio por estado, usuarios y oficinas); un usuario de oficina obtiene las cifras **acotadas a su oficina** (`countByOficinaId`, `countByEstadoGroupedByOficina`); cualquier otro rol recibe **403**. Resuelve los conteos por estado con los *counts agrupados* (una sola consulta `GROUP BY` por dominio) y adjunta los próximos 10 eventos reutilizando `SolicitudEventoServiceImpl.listarProximos`.

AuditoriaServiceImpl — consulta del **historial de cambios** con Hibernate Envers. Expone `historialUsuarios`, `historialSolicitudesEvento` e `historialSolicitudesAnuncio`. El método genérico `historial` obtiene un `AuditReader` del `EntityManager`, verifica que la entidad esté auditada (`isEntityClassAudited`, si no **400**), consulta todas las revisiones de la fila (`forRevisionsOfEntity`) y las mapea a `AuditoriaRevisionResponse` con el número de revisión, la fecha, el **tipo de cambio** (`ADD` / `MOD` / `DEL`) y un resumen legible, ordenadas de la más reciente a la más antigua. Si no hay revisiones, responde **404**.

2.8 Capa de controllers (referencia de endpoints)

Los controllers (`controller/`) constituyen la capa de presentación HTTP de GEA. Todos están anotados con `@RestController` y, salvo cuando exponen rutas heterogéneas, declaran su prefijo común con `@RequestMapping`. Todas las rutas que se listan a continuación son **relativas al `context-path` `/api`** (puerto 8083): por ejemplo `POST /auth/login` se invoca realmente como `POST /api/auth/login`.

El patrón es uniforme: el controller recibe la petición, valida el cuerpo entrante con `@Valid` sobre un *DTO Request*, delega en el servicio correspondiente y devuelve el resultado envuelto en `ApiResponse<T>` dentro de un `ResponseEntity`. No contienen lógica de negocio.

La **autorización** se aplica en dos niveles complementarios: a nivel de URL mediante `SecurityConfig` (cadena de filtros) y a nivel de método mediante la anotación `@PreAuthorize("hasAnyRole(...)")`. Los roles del sistema son `SUPER_ADMIN`, `ADMIN`, `COMUNICACIONES`, `OFICINA` y `USUARIO_AUTENTICADO_APP`. En las tablas siguientes, la columna **Roles** indica los roles exigidos por `@PreAuthorize`; cuando aparece (*público / autenticado*) significa que el método no declara `@PreAuthorize` propio y queda regido únicamente por la regla de `SecurityConfig` para esa ruta.

A continuación se documenta cada uno de los 12 controllers con su ruta base y la lista completa de endpoints reales.

2.8.1 AuthController (`/auth`)

Autenticación de usuarios. No exige roles: son los puntos de entrada para obtener el token JWT.

Método	Ruta	Roles	Request	Response	Descripción
POST	<code>/auth/login</code>	(público)	<code>AuthRequest</code>	<code>AuthResponse</code>	Login por correo/contraseña; devuelve el JWT y los datos básicos del usuario.
POST	<code>/auth/microsoft/mobile</code>	(público)	<code>MicrosoftAuthRequest</code>	<code>AuthResponse</code>	Login federado con Microsoft (app móvil); valida el <code>idToken</code> y emite el JWT propio de GEA.

2.8.2 UsuarioController (`/admin/usuarios`)

CRUD de usuarios del panel administrativo. Restringido a administradores.

Método	Ruta	Roles	Request	Response	Descripción
POST	<code>/admin/usuarios</code>	SUPER_ADMIN, ADMIN	<code>UsuarioRequest</code>	<code>UsuarioResponse</code>	Crea un usuario (HTTP 201). Valida unicidad de correo/teléfono y encripta la contraseña.
GET	<code>/admin/usuarios/{id}</code>	SUPER_ADMIN, ADMIN	—	<code>UsuarioResponse</code>	Obtiene un usuario por id.
PUT	<code>/admin/usuarios/{id}</code>	SUPER_ADMIN, ADMIN	<code>UsuarioRequest</code>	<code>UsuarioResponse</code>	Actualiza un usuario; invalida su entrada en la caché de seguridad.
GET	<code>/admin/usuarios</code>	SUPER_ADMIN, ADMIN	query: q, rol, estado	<code>List<UsuarioResponse></code>	Lista usuarios con filtros opcionales por texto, rol y estado.
DELETE	<code>/admin/usuarios/{id}</code>	SUPER_ADMIN, ADMIN	—	<code>Void</code>	Elimina un usuario.

2.8.3 OficinaController (`/admin/oficinas`)

Gestión de oficinas/programas.

Método	Ruta	Roles	Request	Response	Descripción
POST	<code>/admin/oficinas</code>	SUPER_ADMIN, ADMIN	<code>OficinaRequest</code>	<code>OficinaResponse</code>	Crea una oficina/programa (HTTP 201).
GET	<code>/admin/oficinas</code>	SUPER_ADMIN, ADMIN, COMUNICACIONES, OFICINA, USUARIO_AUTENTICADO_APP	—	<code>List<OficinaResponse></code>	Lista las oficinas activas.

2.8.4 TipoEventoController (`/usuario/tipos-evento`)

Catálogo de tipos de evento (con color).

Método	Ruta	Roles	Request	Response	Descripción
GET	<code>/usuario/tipos-evento</code>	(autenticado)	—	<code>List<TipoEventoResponse></code>	Lista los tipos de evento activos.
POST	<code>/usuario/tipos-evento</code>	SUPER_ADMIN, ADMIN, COMUNICACIONES	<code>TipoEventoRequest</code>	<code>TipoEventoResponse</code>	Crea un tipo de evento (HTTP 201).

2.8.5 LugarFisicoController (/lugares-fisicos)

Catálogo de lugares físicos.

Método	Ruta	Roles	Request	Response	Descripción
GET	/lugares-fisicos	(autenticado)	—	List<LugarFisicoResponse>	Lista los lugares físicos activos.

2.8.6 DispositivoUsuarioController (/usuario/dispositivos)

Registro de tokens FCM (notificaciones push) del usuario autenticado. Usa el `Authentication` del contexto para identificar al dueño del token.

Método	Ruta	Roles	Request	Response	Descripción
POST	/usuario/dispositivos/registran	(autenticado)	DispositivoTokenRequest	Void	Registra/reasigna un token FCM al usuario actual (<i>upsert</i>).
POST	/usuario/dispositivos/desregistrar	(autenticado)	DispositivoTokenRequest	Void	Elimina un token FCM del usuario actual.

2.8.7 ArchivoController (rutas heterogéneas)

Subida y descarga de archivos. No declara prefijo común (`@RequestMapping` sin valor); cada método define su ruta absoluta.

Método	Ruta	Roles	Request	Response	Descripción
POST	/comunicaciones/archivos/upload	SUPER_ADMIN, COMUNICACIONES, ADMIN, OFICINA, USUARIO_AUTENTICADO_APP	multipart/form-data (archivo)	ArchivoResponse	Sube un archivo y devuelve sus metadatos (incluido el token de acceso público).
GET	/archivos/public/{tokenAcceso}	(público)	—	Resource (binario)	Descarga pública del archivo por token; detecta el Content-Type y aplica defensa <i>path traversal</i> .

2.8.8 SolicitudEventoController (rutas heterogéneas)

Ciclo de vida completo de las solicitudes de evento: creación por el solicitante, revisión por Comunicaciones, publicación, series recurrentes y consulta pública de eventos publicados. Sin prefijo común; agrupa rutas bajo `/oficina/...`, `/comunicaciones/...` y `/app/...`.

Método	Ruta	Roles	Request	Response	Descripción
POST	/oficina/solicitudes-evento	SUPER_ADMIN, ADMIN, OFICINA, USUARIO_AUTENTICADO_APP, COMUNICACIONES	SolicitudEventoRequest	SolicitudEventoResponse	Crea una solicitud de evento (HTTP 201).
GET	/oficina/solicitudes-evento	SUPER_ADMIN, ADMIN, OFICINA, USUARIO_AUTENTICADO_APP, COMUNICACIONES	query: q , estado , mes , anio	List<SolicitudEventoResponse>	Lista las solicitudes propias del usuario, con filtros.
GET	/oficina/solicitudes-evento/{id}	SUPER_ADMIN, ADMIN, OFICINA, USUARIO_AUTENTICADO_APP, COMUNICACIONES	—	SolicitudEventoResponse	Obtiene una solicitud propia.
PUT	/oficina/solicitudes-evento/{id}	SUPER_ADMIN, ADMIN, OFICINA, USUARIO_AUTENTICADO_APP, COMUNICACIONES	SolicitudEventoRequest	SolicitudEventoResponse	Actualiza una solicitud propia.
DELETE	/oficina/solicitudes-evento/{id}	SUPER_ADMIN, ADMIN, OFICINA, USUARIO_AUTENTICADO_APP, COMUNICACIONES	—	Void	Elimina una solicitud propia.
DELETE	/oficina/solicitudes-evento/serie/{idGrupo}	SUPER_ADMIN, ADMIN, OFICINA, USUARIO_AUTENTICADO_APP, COMUNICACIONES	—	Void	Elimina la serie recurrente propia.
GET	/comunicaciones/solicitudes-evento	SUPER_ADMIN, COMUNICACIONES, ADMIN	query: q , estado , mes , anio	List<SolicitudEventoResponse>	Bandeja de revisión: lista todas

Método	Ruta	Roles	Request	Response	Descripción
					las solicitudes.
GET	/comunicaciones/solicitudes-evento/{id}	SUPER_ADMIN, COMUNICACIONES, ADMIN	—	SolicitudEventoResponse	Obtiene una solicitud para revisión.
POST	/comunicaciones/solicitudes-evento/{id}/aprobar	SUPER_ADMIN, COMUNICACIONES, ADMIN	—	SolicitudEventoResponse	Aprueba la solicitud.
POST	/comunicaciones/solicitudes-evento/{id}/rechazar	SUPER_ADMIN, COMUNICACIONES, ADMIN	RechazoRequest	SolicitudEventoResponse	Rechaza la solicitud con motivo.
POST	/comunicaciones/solicitudes-evento/{id}/publicar	SUPER_ADMIN, COMUNICACIONES, ADMIN	PublicacionEventoRequest	PublicacionEventoResponse	Publica el evento aprobado.
POST	/comunicaciones/solicitudes-evento/serie/{idGrupo}/aprobar	SUPER_ADMIN, COMUNICACIONES, ADMIN	—	Void	Aprobación masiva de una serie recurrente.
POST	/comunicaciones/solicitudes-evento/serie/{idGrupo}/publicar	SUPER_ADMIN, COMUNICACIONES, ADMIN	PublicacionEventoRequest	Void	Publicación masiva de una serie recurrente.
DELETE	/comunicaciones/solicitudes-evento/serie/{idGrupo}	SUPER_ADMIN, COMUNICACIONES, ADMIN	—	Void	Elimina una serie recurrente completa.
GET	/app/eventos/publicados	(público)	query: filtro, fecha, q, tipoEvento, mes, anio	List<PublicacionEventoResponse>	Lista de eventos publicados (consumo de la app).
GET	/app/eventos/publicados/{id}	(público)	—	PublicacionEventoResponse	Detalle de un evento publicado.
GET	/app/eventos/proximos	(público)	query: limit (1-50, def. 3), q	List<PublicacionEventoResponse>	Próximos eventos.
PATCH	/comunicaciones/eventos-publicados/{id}/visibilidad	SUPER_ADMIN, COMUNICACIONES, ADMIN	query: visible	PublicacionEventoResponse	Alterna la visibilidad de un evento publicado.
PUT	/comunicaciones/eventos-publicados/{id}	SUPER_ADMIN, COMUNICACIONES, ADMIN	UpdatePublicacionRequest	PublicacionEventoResponse	Edita un evento publicado.
DELETE	/comunicaciones/eventos-publicados/{id}	SUPER_ADMIN, COMUNICACIONES, ADMIN	—	Void	Elimina un evento publicado.
GET	/app/eventos/agenda/export/pdf	(público)	query: desde, hasta	byte[] (PDF)	Exporta la agenda de eventos a PDF.

2.8.9 SolicitudAnuncioController (rutas heterogéneas)

Ciclo de vida de las solicitudes de anuncio, paralelo al de eventos. Sin prefijo común; agrupa rutas bajo /app/... y /comunicaciones/...

Método	Ruta	Roles	Request	Response	Descripción
POST	/app/solicitudes-anuncio	USUARIO_AUTENTICADO_APP, OFICINA, SUPER_ADMIN, COMUNICACIONES, ADMIN	SolicitudAnuncioRequest	SolicitudAnuncioResponse	Crea una solicitud de anuncio (HTTP 201).

Método	Ruta	Roles	Request	Response	Descripción
GET	/app/solicitudes-anuncio/mis-solicitudes	USUARIO_AUTENTICADO_APP, OFICINA, SUPER_ADMIN, COMUNICACIONES	query: q , estado , mes , anio	List<SolicitudAnuncioResponse>	Lista las solicitudes propias.
DELETE	/app/solicitudes-anuncio/{id}	SUPER_ADMIN, ADMIN, OFICINA, USUARIO_AUTENTICADO_APP, COMUNICACIONES	—	Void	Elimina una solicitud propia.
PUT	/app/solicitudes-anuncio/{id}	SUPER_ADMIN, ADMIN, OFICINA, USUARIO_AUTENTICADO_APP, COMUNICACIONES	SolicitudAnuncioRequest	SolicitudAnuncioResponse	Actualiza una solicitud propia.
GET	/comunicaciones/solicitudes-anuncio	SUPER_ADMIN, COMUNICACIONES, ADMIN	query: q , estado , mes , anio	List<SolicitudAnuncioResponse>	Bandeja de revisión de anuncios.
GET	/comunicaciones/solicitudes-anuncio/{id}	SUPER_ADMIN, COMUNICACIONES, ADMIN	—	SolicitudAnuncioResponse	Obtiene una solicitud para revisión.
POST	/comunicaciones/solicitudes-anuncio/{id}/aprobar	SUPER_ADMIN, COMUNICACIONES, ADMIN	—	SolicitudAnuncioResponse	Aprueba la solicitud.
POST	/comunicaciones/solicitudes-anuncio/{id}/rechazar	SUPER_ADMIN, COMUNICACIONES, ADMIN	RechazoRequest	SolicitudAnuncioResponse	Rechaza la solicitud con motivo.
POST	/comunicaciones/solicitudes-anuncio/{id}/publicar	SUPER_ADMIN, COMUNICACIONES, ADMIN	PublicacionAnuncioRequest	PublicacionAnuncioResponse	Publica el anuncio aprobado.
GET	/app/anuncios/publicados	(público)	query: q , categoria , mes , anio	List<PublicacionAnuncioResponse>	Lista los anuncios publicados (consumo de la app).
GET	/app/anuncios/publicados/{id}	(público)	—	PublicacionAnuncioResponse	Detalle de un anuncio publicado.
PATCH	/comunicaciones/anuncios-publicados/{id}/visibilidad	SUPER_ADMIN, COMUNICACIONES, ADMIN	query: visible	PublicacionAnuncioResponse	Alterna la visibilidad de un anuncio publicado.
PUT	/comunicaciones/anuncios-publicados/{id}	SUPER_ADMIN, COMUNICACIONES, ADMIN	UpdatePublicacionRequest	PublicacionAnuncioResponse	Edita un anuncio publicado.
DELETE	/comunicaciones/anuncios-publicados/{id}	SUPER_ADMIN, COMUNICACIONES, ADMIN	—	Void	Elimina un anuncio publicado.

2.8.10 ReporteController (/reportes)

Generación, consulta y exportación de reportes de solicitudes, además de las estadísticas del dashboard analítico.

Método	Ruta	Roles	Request	Response	Descripción
POST	/reportes/solicitudes	COMUNICACIONES, SUPER_ADMIN, ADMIN, OFICINA, USUARIO_AUTENTICADO_APP	GenerarReporteRequest	ReporteGeneradoResponse	Genera y per un reporte.
GET	/reportes/solicitudes	COMUNICACIONES, SUPER_ADMIN, ADMIN, OFICINA, USUARIO_AUTENTICADO_APP	query: idOficina , desde , hasta	List<ReporteGeneradoResponse>	Lista los repc generados, c filtros.

Método	Ruta	Roles	Request	Response	Descripción
GET	/reportes/solicitudes/resumen	COMUNICACIONES, SUPER_ADMIN, ADMIN, OFICINA, USUARIO_AUTENTICADO_APP	query: desde , hasta	ReporteSolicitudesResponse	Resumen de solicitudes por rango de fechas.
GET	/reportes/dashboard	COMUNICACIONES, SUPER_ADMIN	query: idOficina , desde , hasta , tipo	ReporteDashboardDTO	KPIs y series de dashboard analítico.
GET	/reportes/solicitudes/export/xlsx	COMUNICACIONES, SUPER_ADMIN, ADMIN	query: desde , hasta	byte[] (XLSX)	Exporta el reporte a Excel (rango máximo 366 días).
GET	/reportes/solicitudes/export/pdf	COMUNICACIONES, SUPER_ADMIN, ADMIN	query: desde , hasta	byte[] (PDF)	Exporta el reporte a PDF (rango máximo 366 días).
GET	/reportes/solicitudes/{id}/export	COMUNICACIONES, SUPER_ADMIN, ADMIN, OFICINA, USUARIO_AUTENTICADO_APP	—	byte[] (PDF/XLSX)	Descarga un reporte guardado en su formato original.
PUT	/reportes/solicitudes/{id}	COMUNICACIONES, SUPER_ADMIN, ADMIN, OFICINA, USUARIO_AUTENTICADO_APP	ActualizarReporteRequest	ReporteGeneradoResponse	Edita el nombre/descripción de un reporte guardado.

2.8.11 DashboardController (/dashboard)

Resumen operativo del panel.

Método	Ruta	Roles	Request	Response	Descripción
GET	/dashboard/resumen	SUPER_ADMIN, COMUNICACIONES, ADMIN, OFICINA	—	DashboardResumenResponse	Métricas y próximos eventos; el alcance (global vs. oficina) depende del rol.

2.8.12 AuditoriaController (/admin/auditoria)

Consulta del historial de cambios (Hibernate Envers).

Método	Ruta	Roles	Request	Response	Descripción
GET	/admin/auditoria/usuarios/{id}	SUPER_ADMIN, ADMIN, COMUNICACIONES	—	List<AuditoriaRevisionResponse>	Historial de revisiones de un usuario.
GET	/admin/auditoria/solicitudes-evento/{id}	SUPER_ADMIN, ADMIN, COMUNICACIONES	—	List<AuditoriaRevisionResponse>	Historial de revisiones de una solicitud de evento.
GET	/admin/auditoria/solicitudes-anuncio/{id}	SUPER_ADMIN, ADMIN, COMUNICACIONES	—	List<AuditoriaRevisionResponse>	Historial de revisiones de una solicitud de anuncio.

2.9 DTOs (objetos de transferencia)

Los DTOs (`dto/request/` y `dto/response/`) separan el modelo interno (entidades JPA) del contrato externo de la API. Los **Request DTOs** validan la entrada con Bean Validation (`@NotNull` , `@NotBlank` , `@Email` , `@Size` , `@Pattern` , `@FutureOrPresent` , etc.) antes de que el dato llegue a la capa de negocio; los **Response DTOs** moldean la salida, evitando exponer campos sensibles (como el `password`) y desacoplando el esquema de base de datos del JSON público. Casi todos usan Lombok: `@Data` en los request y `@Data @Builder` en los response.

2.9.1 DTOs de request (17)

DTO	Usado por	Campos principales (validaciones)
AuthRequest	POST /auth/login	correo (@NotBlank , @Email) , password (@NotBlank) .
LoginRequest	(variante de login)	correo (@NotBlank) , password (@NotBlank) .
MicrosoftAuthRequest	POST /auth/microsoft/mobile	idToken (@NotBlank) .
UsuarioRequest	UsuarioController (crear/actualizar)	nombre (@NotBlank , 2–120) , correo (@NotBlank , @Email , ≤120) , telefono (≤20) , password (6–100) , idRol , rol , idOficina , authProvider , microsoftOid , fotoUrl , estado .

DTO	Usado por	Campos principales (validaciones)
OficinaRequest	OficinaController (crear)	nombre (@NotBlank, ≤150), programaAcademico (≤200), descripcion (≤500), activa.
TipoEventoRequest	TipoEventoController (crear)	nombre (@NotBlank), descripcion, colorHex (@NotBlank, @Pattern hex #RRGGBB), activo.
DispositivoTokenRequest	DispositivoUsuarioController	token FCM (@NotBlank).
SolicitudEventoRequest	SolicitudEventoController (crear/actualizar)	nombreEvento (@NotBlank), fechaEvento (@NotNull, @FutureOrPresent), horaInicio / horaFin (@NotNull), tipoEvento (@NotBlank), idsLugaresFisicos, linkConexion, responsableEvento, frecuencia (frecuenciaRecurrencia, fechaFinRecurrencia), flags (requiereTransmision, requiereCubrimiento, esImportante, requierePiezaGrafica), idOficina, participantes (@Valid, lista de SolicitudEventoParticipanteRequest).
SolicitudEventoParticipanteRequest	anidado en SolicitudEventoRequest	nombre (@NotBlank), cargo, descripcion, fotoUrl, telefono, correo, tipo (@NotNull, TipoParticipante).
SolicitudAnuncioRequest	SolicitudAnuncioController (crear/actualizar)	titulo (@NotBlank), descripcion, categoria, idsLugaresFisicos, correoContacto, responsableAnuncio, fechaInicioPublicacion / fechaFinPublicacion, horaInicio / horaFin, piezaGraficaUrl, requierePiezaGrafica.
RechazoRequest	endpoints .../rechazar (evento y anuncio)	motivo (@NotBlank).
PublicacionEventoRequest	endpoints .../publicar de evento/serie	tituloVisible (@NotBlank), descripcionVisible, piezaGraficaUrl, fechaPublicacion.
PublicacionAnuncioRequest	endpoint .../publicar de anuncio	tituloVisible (@NotBlank), descripcionVisible, piezaGraficaUrl, fechaPublicacion.
UpdatePublicacionRequest	edición de publicaciones de evento/anuncio	DTO unificado para editar publicaciones de ambos dominios: campos visibles (tituloVisible ≤200, descripcionVisible, piezaGraficaUrl, idsLugaresFisicos), campos de evento (nombreEvento, fechaEvento, horas, tipoEvento, flags), campos de anuncio (titulo, categoria, fechas de publicación, contacto), idOficina, participantes.
GenerarReporteRequest	POST /reportes/solicitudes	nombre (@NotBlank), descripcion, formato (@NotBlank), alcance (@NotBlank), desde / hasta (@NotNull), idOficina, idTipoEvento.
ActualizarReporteRequest	PUT /reportes/solicitudes/{id}	nombre, descripcion (sin validaciones).
CorreoPruebaRequest	utilidad de envío de correo de prueba	destinatario (@Email, @NotBlank), titulo, lugar, oficina.

2.9.2 DTOs de response (17)

DTO	Devuelto por	Campos principales
AuthResponse	endpoints de AuthController	token (JWT), nombre, correo, rol, idOficina, oficinaNombre, fotoUrl, tipo (def. "Bearer").
UsuarioResponse	UsuarioController	id, nombre, correo, telefono, estado, rol, idOficina, oficinaNombre, authProvider, fotoUrl, fechas y usuarios de auditoría.
OficinaResponse	OficinaController	id, nombre, programaAcademico, descripcion, activa.
TipoEventoResponse	TipoEventoController	id, nombre, descripcion, colorHex, activo.
LugarFisicoResponse	LugarFisicoController	id, nombre, descripcion, capacidad, activo.
ArchivoResponse	ArchivoController (upload)	id, nombreArchivo, nombreOriginal, tokenAcceso, url, contentType, tamaño.
SolicitudEventoResponse	SolicitudEventoController	datos del evento, estado, motivoRechazo, lugares (lugar / lugares / idsLugaresFisicos), tipo de evento (id/nombre/color), oficina, solicitante, recurrencia (frecuenciaRecurrencia, idGrupoRecurrencia, esPrincipal), visible, auditoría y participantes.
SolicitudEventoParticipanteResponse	anidado en SolicitudEventoResponse	id, nombre, cargo, descripcion, fotoUrl, telefono, correo, tipo.
SolicitudAnuncioResponse	SolicitudAnuncioController	datos del anuncio, estado, motivoRechazo, lugares, contacto, fechas/horas de publicación, visible, oficina, solicitante (id/correo/nombre) y auditoría.

DTO	Devuelto por	Campos principales
PublicacionEventoResponse	publicaciones de evento (app/comunicaciones)	id, solicitudEventoId, campos visibles (tituloVisible, descripcionVisible, piezaGraficaUrl), datos del evento, lugares, tipo de evento (id/nombre/color), oficina, responsableEvento, flags y visible.
PublicacionAnuncioResponse	publicaciones de anuncio (app/comunicaciones)	id, solicitudAnuncioId, campos visibles, categoria, lugares, contacto, fechas/horas, oficinaNombre, fechaPublicacion, visible, requierePiezaGrafica.
DashboardResumenResponse	DashboardController	alcance y conteos (eventos/anuncios pendientes/aprobados/publicados, totales de usuarios y oficinas) + proximosEventos (List<PublicacionEventoResponse>).
ReporteGeneradoResponse	ReporteController	id, nombre, descripcion, formato, desde / hasta, alcance, fechaCreacion, datos del usuario generador, idOficina, idTipoEvento.
ReporteSolicitudesResponse	GET /reportes/solicitudes/resumen	alcance, rango de fechas, totales y conteos por estado de eventos y anuncios + solicitudes (List<SolicitudResumenDTO>).
ReporteDashboardDTO	GET /reportes/dashboard	KPIs (totalSolicitudes, aprobados/pendientes/rechazados, tasaAprobacion) y series de gráficas (eventosPorTipo, solicitudesPorMes, solicitudesPorOficina, tendenciaEstado) + solicitudes.
SolicitudResumenDTO	anidado en reportes	id, tipo (EVENTO/ANUNCIO), titulo, oficina, fechaRegistro, estado.
AuditoriaRevisionResponse	AuditoriaController	revision, fechaRevision, tipoCambio (ADD/MOD/DEL), resumen legible.

Nota: ReporteDashboardDTO referencia además GrupoDatoDTO (paquete dto/common/) como elemento de las series de gráficas.

2.9.3 ApiResponse (envoltorio estándar)

Todas las respuestas JSON de la API se envuelven en el tipo genérico `ApiResponse<T>` (paquete `util/`), lo que garantiza un contrato uniforme para el cliente. Su estructura es:

Campo	Tipo	Significado
success	boolean	true en respuestas correctas, false en errores.
message	String	Mensaje legible (p. ej. "Usuario creado exitosamente").
data	T	Carga útil: el DTO de respuesta, una lista o null (en operaciones sin retorno como los DELETE).
timestamp	LocalDateTime	Momento de generación de la respuesta.

Ofrece *factory methods* estáticos que los controllers usan de forma sistemática: `ApiResponse.success(data, message)`, `ApiResponse.success(data)` (mensaje por defecto "Operación exitosa") y `ApiResponse.error(message)`. Las respuestas de error se generan de forma centralizada en `GlobalExceptionHandler` (`@RestControllerAdvice`), manteniendo el mismo formato. Las descargas binarias (PDF/XLSX y archivos públicos) son la excepción: devuelven `byte[] / Resource` directamente, sin envoltorio, porque su `Content-Type` no es JSON.

2.10 Mappers

Los 5 mappers (`mapper/`) traducen entre **entidades JPA y DTOs de respuesta**. Todos están implementados con **MapStruct** (`@Mapper(componentModel = "spring")`): son interfaces y MapStruct genera la implementación en tiempo de compilación, registrándola como bean de Spring para inyectarla en los servicios. La conversión es **unidireccional** (entidad → *DTO Response*); la dirección inversa (Request → entidad) se realiza manualmente en la capa de servicio, donde requiere resolver relaciones (oficina, lugares, tipo de evento) contra la base de datos. Para campos derivados o aplanados, los mappers usan `@Mapping(source = ...)` (navegación de relaciones) y `@Mapping(expression = "java(...)")` (lógica inline, p. ej. concatenar nombres de lugares o resolver la oficina con *fallbacks*).

Mapper	Entidad → DTO	Qué convierte / detalles
LugarFisicoMapper	LugarFisico → LugarFisicoResponse	MapStruct. Mapeo directo campo a campo (uno y lista). El más simple, sin @Mapping explícitos.
SolicitudEventoMapper	SolicitudEvento → SolicitudEventoResponse (y SolicitudEventoParticipante → SolicitudEventoParticipanteResponse)	MapStruct. Aplana tipo de evento (tipoEventoCatalogo → id/nombre/colorHex), oficina, solicitante y lista de participantes; concatena lugares con expresiones Java; fija visible = false.
SolicitudAnuncioMapper	SolicitudAnuncio → SolicitudAnuncioResponse	MapStruct. Resuelve solicitante y oficina con <i>fallbacks</i> por expresión (usa contacto/responsable si no hay usuario; ignora la oficina para USUARIO_AUTENTICADO_APP); concatena lugares; fija visible = false.

Mapper	Entidad → DTO	Qué convierte / detalles
PublicacionEventoMapper	PublicacionEvento → PublicacionEventoResponse	MapStruct. Toma los campos del evento desde la <code>solicitudEvento</code> asociada (fecha, horas, lugares, tipo, oficina, flags, solicitante).
PublicacionAnuncioMapper	PublicacionAnuncio → PublicacionAnuncioResponse	MapStruct. Toma los campos del anuncio desde la <code>solicitudAnuncio</code> asociada; resuelve la oficina con <i>fallback</i> al solicitante; concatena lugares.